MDPI

Article

Enhancing Consent Policy Privacy in Blockchain-Based Healthcare Data Sharing

Abstract

Blockchain-based consent management makes healthcare data sharing auditable, but on a public blockchain, enforcement can reveal the consent policies. These policies encode conditions such as disease-specific research restrictions, whose exposure can disclose sensitive information about the data subject. To address this, we propose TPS-ZKPI, in which a Trusted Proof Service (TPS) generates zero-knowledge proofs of compliance that a smart contract verifies against committed policies. In TPS-ZKPI, we compile each policy into a fixed-length program for a reusable zero-knowledge policy interpreter, so all policies share one verification interface, and mask residual gas differences with randomized gas padding. We evaluate TPS-ZKPI on Polygon Amoy with supervised and unsupervised attacks in which a passive public-chain observer tries to infer hidden policy properties from public-chain metadata. The results show that 43 of 45 supervised tasks stay below our leakage criterion, and both failures trace to chance correlations with the padding. Furthermore, an ablation shows that without padding the same attacks often recover policy content and structure, while padding adds about 46% gas to access and 38% to policy commitment. Overall, this paper shows empirically that consent enforcement on a public blockchain can resist the modeled metadata attacks, provided the TPS is trusted.

Keywords: blockchain; consent policy privacy; zero-knowledge proofs; Data Use Ontology (DUO); healthcare data sharing; metadata leakage

1. Introduction

Blockchain technology has emerged as a promising foundation for decentralized and auditable healthcare data sharing [1–3]. Its transparency and immutability allow consent enforcement and authorization decisions to be verified and traced across institutions. However, the same transparency means that consent enforcement on a public blockchain can expose information about the governing consent policies, even when the health data themselves remain encrypted [4,5].

This exposure matters because consent policies in healthcare are often privacy-sensitive [6]. A *consent policy* encodes the conditions under which a data subject authorizes others to access or use their health data, such as who may access the data, for what purpose, and under what constraints. For example, a policy based on biomedical data-use terms may reveal whether a dataset is restricted to disease-specific research, whether ethics approval is required, or whether access is limited to particular institutions or projects. If such a policy is written into a smart contract, embedded in verification logic, or reflected in public execution traces, unauthorized observers may infer information about the protected resource, the data subject's constraints, or the requester's authorization context.

Received:
Accepted:
Published:

Copyright: © 2026 by the authors. Submitted to *Blockchains* for possible open access publication under the terms and conditions of the Creative Commons Attribution (CC BY) license.

Existing privacy-preserving blockchain solutions do not address this problem, because they mainly protect data contents, user identities, or transaction details [7–9]. Encrypted data-sharing mechanisms such as attribute-based encryption (ABE) and proxy re-encryption control access to data [10,11], and zero-knowledge transaction systems hide asset ownership, values, or participant relationships [12]; none of them protects the confidentiality of the policies that govern access decisions. Hidden-policy attribute-based encryption [13,14] does hide policy content, but it does not control what a public chain exposes while it enforces the policy. On a public chain, a policy can leak not only through explicit disclosure but also through metadata: the size of the calldata, the layout of the public inputs, the emitted events, the verifier interface, and the gas consumed. Policy-private consent enforcement therefore requires more than hiding data or proving correctness in zero knowledge: it must also control this metadata leakage, that is, what the public-chain transcript (the transaction data, events, and gas visible to any observer) reveals about the hidden policy.

This paper studies this problem for blockchain-based healthcare data sharing on public blockchains. The challenge is to let an on-chain verifier check that a requester satisfies a provider-defined consent policy, while ensuring that observers who see only public-chain metadata learn minimal information about the governing policy and the requester's authorization evidence. We focus on Boolean consent policies, including policies derived from biomedical data-use conditions such as terms from the Data Use Ontology (DUO), the Global Alliance for Genomics and Health (GA4GH) standard vocabulary of consent-based data-use conditions [6], and disease-scope constraints [15].

To address this problem, we propose the Trusted Proof Service-assisted Zero-Knowledge Policy Interpreter (TPS-ZKPI). In TPS-ZKPI, the data provider commits to a hidden consent policy on-chain and provisions the corresponding policy-bound proving material to a Trusted Proof Service (TPS). A requester submits its policy-relevant authorization evidence, such as attributes, credentials, and access context, to the TPS, which constructs the private witness and generates a zero-knowledge proof that the requester satisfies the committed policy. A smart contract then verifies this proof against the provider's policy commitment. As a result, every authorization decision remains publicly auditable, while the policy logic never appears in the public-chain transcript.

To prevent the transcript itself from revealing the policy, TPS-ZKPI combines four design elements. First, proof generation at the TPS keeps confidentiality-sensitive policy material and requester evidence off the chain. Second, heterogeneous Boolean consent policies are compiled into fixed-length Reverse Polish Notation (RPN) programs, so that every policy has the same length. Third, building on the standard commit-and-prove paradigm, a reusable zero-knowledge policy interpreter evaluates these programs; this avoids policy-specific verifier contracts and allows policies to evolve without changing the public verification interface. Fourth, transcript normalization, including randomized gas padding, reduces the remaining policy-dependent variation in calldata, event logs, public parameters, and gas consumption.

We evaluate TPS-ZKPI against a passive public-chain observer who sees only the public-chain transcript. The evaluation expresses consent policy privacy as three objectives: Policy Content Hiding (PCH), Policy Indistinguishability (PI), and Structure and Parameter Privacy (SPP). For these objectives, supervised and unsupervised attacks test whether the public-chain metadata allow the observer to recover hidden policy content, to distinguish otherwise-equivalent policies, or to infer structural and parameter properties of a policy. The results show that 43 of the 45 supervised tasks stay below our leakage criterion, which flags a task only when the attack exceeds chance by a fixed margin and is statistically significant in at least half of 30 policy-disjoint splits; both failures trace to chance

correlations with the randomly drawn padding targets. Furthermore, a gas-padding ablation shows that, without padding, the same attacks recover a policy-correlated gas signal, which randomized gas padding suppresses.

This paper makes the following contributions:

- We propose TPS-ZKPI, a TPS-assisted zero-knowledge approach to publicly verifiable and auditable consent enforcement. On-chain proof verification and auditability hold without trusting the TPS, whereas policy confidentiality and faithful witness construction rely on an explicit, delimited trust assumption on the TPS, which keeps policy-bound proving material and requester authorization evidence off-chain.
- We encode each Boolean consent policy as a fixed-length RPN program over a bounded token alphabet and evaluate it in a single zero-knowledge circuit. Combined with policy commitments, it gives all policies the same policy-independent on-chain verification interface: one circuit serves every policy, and policies can evolve without changing the on-chain verifier.
- We empirically evaluate how much policy information remains in the normalized public-chain metadata on the Polygon Amoy testnet[1], using supervised classification attacks judged by our leakage criterion and unsupervised clustering attacks. A paired gas-padding ablation further shows that randomized gas padding is responsible for suppressing the modeled gas channel and quantifies its gas overhead.

The remainder of this paper is organized as follows. Section 2 reviews the technical foundations: cryptographic access control in blockchain healthcare, zero-knowledge proofs, and the Data Use Ontology. Section 3 discusses related work and positions TPS-ZKPI against existing blockchain consent-management, privacy-preserving blockchain, hidden-policy, private policy-evaluation, reusable proving, and metadata-leakage approaches. Section 4 defines the system and trust model, the public-chain adversary, and the privacy objectives. Section 5 presents the TPS-ZKPI design. Section 6 reports the empirical evaluation. Section 7 discusses limitations and future work, and Section 8 concludes the paper.

2. Background

This section reviews the technical foundations relevant to policy-private consent enforcement on blockchains. We first introduce the cryptographic access-control mechanisms commonly used in blockchain-based healthcare data sharing. We then review zero-knowledge proofs and the Data Use Ontology (DUO) [16], which together provide the basis for the policy representation and verification mechanisms used in our approach.

2.1. Cryptographic Access Control in Blockchain Healthcare

Fine-grained consent management in blockchain-based healthcare data sharing is commonly built on cryptographic access-control primitives, chiefly Ciphertext-Policy Attribute-Based Encryption (CP-ABE) and Proxy Re-Encryption (PRE) [11,17,18]. In CP-ABE, the encrypting party embeds an attribute-based access policy into the ciphertext, so that only requesters whose credentials satisfy the policy can decrypt the protected data [10]. PRE lets a semi-trusted intermediary transform a ciphertext encrypted for one party into one decryptable by a designated requester, delegating access without exposing the plaintext [18]. Both primitives remain the building blocks of current blockchain healthcare designs: recent schemes combine PRE with on-chain records to delegate access to medical records and Internet of Medical Things (IoMT) data [19,20], and CP-ABE variants hide the access policy and attributes [13,14], in one case also replacing the single trusted

[1] https://amoy.polygonscan.com/txs?a=0x461853d10fB9A286a75dE05c9B166e77645160D7

authority with distributed key generation [14]. These primitives determine who may obtain the data, but standard CP-ABE does not keep the access policy itself confidential: the policy is attached to the ciphertext in the clear.

2.2. Zero-Knowledge Proofs

Zero-knowledge proofs (ZKPs) are cryptographic protocols that allow a prover to convince a verifier that a statement is true without revealing any information beyond the validity of the statement [21]. Modern ZKP systems support verifiable computation over private inputs, so that complex predicates can be proven without disclosing the underlying data. In blockchain settings, ZKPs are widely used to reconcile public verifiability with privacy: privacy-focused systems such as Zerocash and Tornado Cash apply them to conceal transaction details or provide transaction anonymity [9,22]. These properties make ZKPs a natural building block for proving that a requester satisfies a hidden consent policy, which is the direction our approach develops.

2.3. The Data Use Ontology

DUO provides a standardized vocabulary for expressing biomedical data-sharing consent conditions, and its terms are widely adopted in genomics and biobanking to enable semantic interoperability [6,16]. A DUO-based policy combines permissions and modifiers, such as general research use (GRU), disease-specific research (DS), or a requirement that ethics-committee approval be obtained (IRB), into the condition under which access is granted.

DUO Term Categories

Although DUO offers a rich set of permissions and modifiers, its terms are heterogeneous in how they must be evaluated for compliance. To structure their handling in a zero-knowledge setting, we group DUO terms into four categories:

- **Boolean-only:** simple yes/no conditions that can be represented directly without external validation, e.g. “no restriction” (NRES), general research use (GRU), or “health/medical/biomedical research” (HMB).
- **Ontology-linked:** terms whose evaluation involves consistency checks against external vocabularies or classifications, e.g. disease-specific research (DS), population origins or ancestry research (POA), or clinical care use (CC).
- **Obligation-required:** terms that impose procedural or temporal obligations, e.g. “publication required” (PUB), “publication moratorium” (MOR), “time limit on use” (TS), or “return to database” (RTN).
- **Credential-required:** terms whose evaluation relies on attested proof of requester status or approvals, e.g. “collaboration required” (COL), “ethics approval required” (IRB), “project-specific restriction” (PS), or “institution-specific restriction” (IS).

Table 1 summarizes representative DUO terms under this category. Some terms reduce to simple Boolean flags, while others rely on ontology references, external obligations, or credential attestations.

3. Related Work

Blockchain-based consent management for healthcare data sharing has been studied extensively, with smart contracts used to record, enforce, and audit consent across institutions [1–3]. These works focus on making consent decisions verifiable and auditable, and the privacy mechanisms surveyed in this space mainly protect health data contents and user identities [5]. The confidentiality of the governing consent policy, and what public-chain enforcement artifacts reveal about it, is typically out of scope.

Table 1. Representative DUO terms grouped by evaluation category.

Category	Example Terms	Description
Boolean-only	NRES (DUO:0000004), GRU (DUO:0000042), HMB (DUO:0000006)	Direct yes/no conditions, no external validation required.
Ontology-linked	DS (DUO:0000007), POA (DUO:0000011), CC (DUO:0000043)	Evaluated against external ontologies such as the International Classification of Diseases (ICD-10) or the Mondo Disease Ontology (MONDO).
Obligation-required	PUB (DUO:0000019), MOR (DUO:0000024), TS (DUO:0000025), RTN (DUO:0000029)	Impose obligations such as publication duties, time limits, or data return.
Credential-required	COL (DUO:0000020), IRB (DUO:0000021), PS (DUO:0000027), IS (DUO:0000028)	Depend on credentials, approvals, or institutional/project-specific restrictions.

Beyond consent management, related work addresses individual requirements of policy-private consent enforcement (policy confidentiality, metadata-leakage control, verifiable auditability, or a stable verification interface), but none of the approaches reviewed here combines them on a public chain. Encrypted data-sharing mechanisms, such as CP-ABE and PRE [17,18], control access to encrypted data. However, as noted in Section 2.1, standard CP-ABE exposes the policy itself, while PRE-based delegation may reveal authorization patterns that data encryption is not designed to hide. Transaction-privacy systems, including Zerocash-style protocols [22] and Ethereum-oriented systems such as ZETH and AZTEC [7,8], provide public verification over private transaction witnesses, but they mainly target asset ownership, values, or participant relationships rather than compliance with a provider-defined consent policy.

Hidden-policy access-control work reduces direct policy exposure: CP-ABE variants hide the access structure in the ciphertext [13,14]. Wu et al. [23] instead realize attribute-based access control (ABAC) by encrypting policy and attribute values under an additively homomorphic cryptosystem. A smart contract computes their encrypted difference, one assistant node per organization decrypts it partially with a zero-knowledge proof of correct decryption, and the contract decides without seeing the values; every request therefore needs every organization online. Functional encryption gives a more general cryptographic abstraction for revealing function outputs while hiding inputs [24], but it does not by itself yield publicly verifiable authorization evidence. Private-state blockchain systems, including Secret Network and Oasis/Ekiden, protect smart-contract state or execution through encrypted state and trusted execution environments (TEEs) [25,26]. These lines of work address adjacent forms of policy or state hiding, but TPS-ZKPI targets a more specific setting: hiding consent policies on a public chain while retaining public verifiability of each authorization decision. Even where policies are hidden, existing designs may still leak policy-dependent artifacts through ciphertext structures, policy-update transactions, authorization workflows, or repeated interactions. In the homomorphic design, for example, the Boolean structure of the policy and its attribute names stay on the ledger, and the per-attribute match results can be recomputed from the ciphertexts and partial decryptions recorded with each decision [23]. Moreover, many target permissioned or off-

chain settings, such as that design's coalition blockchain, where enforcement artifacts are not globally visible as they are on public blockchains.

Self-sovereign-identity (SSI) architectures place verifiable claims under the requester's control and raise the converse question of which verifiers the holder can trust with those claims. Buccafurri et al. [27] answer it with ciphertext-policy attribute-based proxy re-encryption. The holder stores encrypted claims off-chain under a self-defined attribute policy, and a semi-trusted, non-colluding consensus provider re-encrypts them for verifiers whose certified attributes satisfy that policy. A smart contract notarizes hashes of storage, policy-change, and access operations so that they can later be attributed to their parties, and a trusted key-generation authority issues all attribute keys. Zero-knowledge credential presentations likewise let a patient authenticate anonymously on public or private chains [28], and surveys of zero-knowledge proofs for blockchain privacy catalogue this design space [12,29]. TPS-ZKPI sits on the other side of the same authorization relation: the protected object is the provider's consent policy rather than the requester's claims, and the claims enter only as private witnesses of a proof with a fixed statement. Two differences follow. First, the smart contract decides rather than notarizes: it verifies the proof and records a context-bound outcome that any observer can check immediately, whereas notarization supports attribution after the fact. Second, off-chain trust is concentrated rather than split: their proxy never sees claim contents, whereas the TPS sees both the policy and the claims, which is why Section 4 treats the TPS as an explicit trust boundary and Table 3 states what survives its compromise. The designs are complementary: the requester could release its claims only to a TPS whose certified attributes satisfy a requester-defined policy, which is the verifier-authorization problem of [27] with the TPS as verifier, and a per-request holder signature over the authorization context would further prevent a TPS from reusing observed evidence for fresh sessions (Section 7).

Privacy-preserving access-control and policy-evaluation mechanisms have been studied in credential-based settings built on the eXtensible Access Control Markup Language (XACML) and the Security Assertion Markup Language (SAML) [30], as well as in policy evaluation by multi-party computation (MPC) using homomorphic encryption (HE) and secure function evaluation [31]. These approaches support confidential authorization computation, but they are not primarily designed for settings where policy-evaluation artifacts (calldata, public inputs, emitted events, timing, and gas usage) are permanently and globally visible on a public blockchain. In addition, these approaches do not directly provide compact evidence that a smart contract can verify against a committed, provider-defined policy instance.

Reusable proving systems, including universal-setup succinct non-interactive arguments of knowledge (SNARKs), composable proof systems, and zero-knowledge virtual machines (zkVMs) [32–34], improve maintainability by reducing the need for policy-specific circuits or verifier contracts. However, when deployed for public-chain consent enforcement, reusable proving alone does not guarantee policy privacy: policy-dependent program representations, public inputs, calldata layouts, logs, or gas profiles may still remain observable. This motivates TPS-ZKPI's use of a domain-specific reusable policy interpreter, which preserves proof reuse while exposing only a policy-independent public verification interface.

Studies of metadata leakage on public chains show that cryptographic privacy guarantees may be weakened by observable transaction-level artifacts such as timing and volume patterns [4]. Techniques such as batching, decoys, and traffic shaping can reduce some observable signals [35], but they are usually designed for payment privacy, anonymity sets, or transaction-linkability settings. They do not directly provide hidden-

policy compliance verification or auditable consent enforcement. In contrast, TPS-ZKPI treats metadata leakage as part of the policy-privacy objective rather than as a separate transaction-level concern.

Existing approaches address verifiable consent recording, data confidentiality, transaction privacy, policy hiding, confidential policy evaluation, proof reuse, or metadata mitigation in isolation, whereas policy-private consent enforcement under public-chain observability requires the combination of policy confidentiality, metadata-leakage control, verifiable auditability, and a stable verification interface for evolving Boolean consent policies. Table 2 summarizes the positioning of our work. Partial indicates limited support or a need for application-specific design; N/A denotes public-chain-specific criteria outside the cited approach's deployment setting.

Table 2. Positioning of representative approaches against policy-private consent enforcement under public-chain observability.

Approach	Policy confidentiality	Metadata-leakage mitigation	Verifiable auditability	Scope and limitations
Blockchain consent management [1,2]	No	No	Yes	Records and enforces consent verifiably on chain, but policy confidentiality and policy-dependent leakage are typically out of scope.
Data / transaction privacy [7,8,17,18,22,36]	No	Partial	Partial	Protects encrypted data access or transaction semantics, but not the governing consent policy or policy-dependent authorization workflow.
Hidden-policy ABE [13,14]	Partial	Partial	Partial	Reduces direct policy exposure, but may leak through workflows, updates, or policy-holder interactions.
Homomorphic hidden-policy ABAC [23]	Partial	No	Yes	Hides policy and attribute values on a permissioned ledger, but the Boolean structure, attribute names, and per-attribute match results remain recoverable from it.
SSI verifier authorization [27]	No	Partial	Partial	Releases the holder's claims only to attribute-qualified verifiers and notarizes access for later attribution, but protects the requester's claims rather than the provider's consent policy.
MPC / HE policy evaluation [31]	Yes	N/A	N/A	Supports confidential policy evaluation off-chain; the cited design does not specify public-chain enforcement or authorization auditing.
Universal SNARKs / zkVMs [32–34]	Partial	Partial	Yes	Supports reusable proving and public verification; policy hiding and a fixed verification interface require application-specific design.
Metadata-leakage defenses [35]	No	Yes	Partial	Mitigates transaction timing and volume leakage and supports verifiable execution, but does not directly provide hidden-policy consent enforcement.
TPS-ZKPI	Yes (under TPS trust)	Yes (modeled channels)	Yes	Requires TPS trust for confidentiality; supports bounded Boolean policies; Metadata mitigation covers the modeled channels.

4. Problem Formulation

4.1. System and Trust Model

We consider a blockchain-based healthcare data-sharing system with two user roles: *data providers*, who provide protected resources and govern access through consent policies, and *data requesters*, who seek access to those resources subject to the applicable policies. TPS-ZKPI mediates this interaction through three components: a TPS operating off-chain, an on-chain access-control manager (ACM), and a data access layer. The provider defines a consent policy P for a protected resource on behalf of the data subject whose

consent it encodes, and registers an on-chain commitment h to the encoded policy. The requester seeks access by submitting policy-relevant authorization evidence, such as attributes, credentials, and access context. The TPS constructs the private witness and generates a zero-knowledge proof of policy compliance. The ACM verifies the proof on-chain and records the authorization outcome, while the data access layer grants access according to the recorded decision.

TPS-ZKPI relies on the following assumptions. First, policies are provider-controlled: the provider is responsible for defining the applicable consent policy and committing the corresponding policy instance on-chain. Second, the TPS is trusted to preserve the confidentiality of policy-bound proving material and requester-submitted authorization evidence, and to follow the prescribed proof-generation workflow. Third, the commitment scheme is assumed to be hiding and binding, and the zero-knowledge proof system is assumed to satisfy completeness, soundness, and zero knowledge with respect to the implemented policy-compliance relation. Issuer governance, credential issuance and revocation, and real-world identity verification are treated as external lifecycle functions.

Conceptually, the TPS is an independent proof service, distinct from the provider; this separation is what makes relocating trust meaningful, since it prevents the provider from being the single party that holds both the policy and the requester evidence. Possible instantiations of such an independent service include a third-party proof service, a TEE-backed proof service that makes the trust assumption hardware-attested, or MPC/-collaborative SNARK proof generation in which no single party sees both policy material and requester evidence. For evaluation simplicity, the current evaluation co-locates the TPS with the provider (a provider-operated TPS). This co-location concentrates trust at the provider side: a single party now holds both the policy and the requester evidence, so a compromised provider-operated TPS could substitute inputs whose origin is not independently authenticated or withhold proof generation from legitimate requesters.

Two guarantees must be distinguished here. Authorization correctness is primarily enforced by the commitment and on-chain proof verification: a proof that does not satisfy the committed policy relation is rejected, largely independently of how the TPS is instantiated. Policy confidentiality, in contrast, is not cryptographically enforced against the TPS itself; it rests entirely on the trust assumption stated above. Our evaluation therefore assumes a TPS that follows the prescribed workflow and isolates the remaining question: what a public-chain observer can infer from the resulting public artifacts. Weakening the TPS trust assumption itself is orthogonal and is the role of the reduced-trust instantiations above. Table 3 summarizes, for each actor or failure mode, which of the four guarantees (policy confidentiality, authorization correctness, liveness, and decision auditability) is enforced cryptographically, which holds only under the TPS trust assumption, and which the prototype leaves to external functions or does not provide.

Attribute-origin guarantees depend on the evidence type. For DUO terms evaluated from the claims in an issuer-signed profile credential presented by the requester (hereafter, profile-backed terms), the prototype verifies the EdDSA-Poseidon credential under an issuer key selected by the provider and bound to the policy commitment. EdDSA-Poseidon denotes the Edwards-curve Digital Signature Algorithm (EdDSA) [37] over the Baby Jubjub curve [38], instantiated with Poseidon [39], a hash function designed for efficient evaluation inside zero-knowledge circuits. The credential binds the policy commitment, requester pseudonym, protected resource, claims, validity interval, and credential identifier; the corresponding policy attributes are derived inside the circuit. Thus, subject to signature unforgeability and Poseidon collision resistance, these values cannot be modified without invalidating the proof. This guarantee does not establish the real-world

Table 3. Guarantees of the TPS-ZKPI prototype against each actor or failure mode, and whether each rests on cryptography, on the TPS following its workflow, or on external functions.

Actor or failure mode	Policy confidentiality	Authorization correctness	Liveness	Decision auditability
Passive public-chain observer	$C_{zk,hid}$ for proof and commitment contents; empirical for modeled metadata	$C_{snd,bind}$	n/a	C_{chain}: outcome recorded
Untrusted requester	As for the observer; adaptive-query leakage not evaluated	$C_{snd,ctx}$: cannot forge a proof or alter its bound authorization context	n/a	C_{chain}: pseudonymous request context and outcome recorded on-chain
Workflow-following TPS	– against the TPS itself; T against onward disclosure	$C_{snd,bind}$ for the committed relation; T/E for unsigned evidence provenance	T	C_{chain} for submitted decisions; – for refusal
Compromised or colluding TPS	–	$C_{snd,bind,sig}$ for the committed relation and signed claims; E for externally authenticated evidence; – for unauthenticated evidence provenance	–: may refuse to prove	C_{chain} for submitted decisions; – for refusal
Forged credential	n/a	$C_{snd,bind,sig}$: signature verified under the policy-bound issuer key	n/a	n/a
Unauthorized or revoked issuer	n/a	E: issuer authorization and revocation status are external	n/a	C_{chain} for the on-chain decision record; E for determining issuer status

C: cryptographic, with subscripts naming the mechanisms it rests on: zk and snd, zero knowledge and soundness of Groth16; hid and bind, hiding and binding of the policy commitment h and the parameter commitment pp; sig, EdDSA-Poseidon signature unforgeability; ctx, on-chain recomputation of the context digest d_χ; chain, integrity of the on-chain record, assuming the underlying chain is secure. T: trust in the TPS following the prescribed workflow; E: external or lifecycle function; –: not provided by the prototype; n/a: not applicable. In the evaluated provider-operated deployment, the provider and TPS are co-located, so all TPS-dependent confidentiality and liveness assumptions are concentrated at the provider.

authority of the issuer or the truthfulness of issuance. Ontology and requester-allowlist predicates likewise establish membership only relative to provider-committed sets.

4.2. Public-Chain Adversary

The primary adversary is a passive public-chain observer. The adversary can inspect smart-contract code and monitor public blockchain artifacts across multiple authorization sessions, including policy commitments, submitted proofs, public inputs, emitted events, calldata, timing, and gas usage. The adversary does not observe provider-to-TPS policy material, commitment-opening randomness, requester-submitted authorization evidence, credential witnesses, or TPS-side witness construction: these items are exchanged over off-chain channels or held locally by the provider and the TPS, and never appear in on-chain artifacts. We assume the off-chain channels themselves are confidential (e.g., TLS-protected).

Each policy-enforcement instance corresponds to an authorization session, in which a requester seeks access to a protected resource under a committed policy. For each such session, the adversary observes a public view

$$view_{\mathsf{pub}} = (h, \pi, pp, d_\chi, \tau, id_{\mathrm{req}}, id_{\mathrm{res}}, \nu, meta), \tag{1}$$

where h is the on-chain commitment to the hidden policy, π is the zero-knowledge proof submitted for on-chain verification, pp, d_χ, and τ are the remaining public signals of the proof statement (a parameter commitment, a context digest, and a request epoch; Section 5), id_{req}, id_{res}, and ν are the requester, resource, and session-nonce values carried in the access transaction, and *meta* denotes transaction-level metadata exposed by the enforcement layer, such as calldata footprint, event structure, public-input layout, timing, and gas usage. The adversary's goal is to infer hidden information about the committed policy or the authorization interaction from this public view.

In addition to the public-chain observer, we treat the requester as untrusted with respect to policy confidentiality. The requester initiates access requests, observes its own submitted evidence and authorization outcome, and inspects the public-chain transcript. However, the requester does not receive the encoded policy, commitment-opening randomness, or policy-bound proving material, which remain within the provider-to-TPS proof-generation boundary. This restriction matters because a requester that learned the policy could infer sensitive facts about the data subject from the consent terms themselves and, in addition, would learn the policy's full acceptance region (which attribute combinations are granted), enabling targeted evidence construction or probing beyond its own legitimate sessions.

The proved authorization outcome is bound to the requester pseudonym, a protected resource, and a session nonce through the context digest defined in Section 5. This binding prevents substitution of the recorded authorization context, but it should not be interpreted as proof of the requester's real-world identity.

4.3. Privacy Objectives

We operationalize consent policy privacy as three measurable, attack-oriented objectives that cover the information a public-chain observer could extract about a hidden policy: its semantic content, which of several similar policies it is, and its structural form. These correspond to Policy Content Hiding, Policy Indistinguishability, and Structure and Parameter Privacy, respectively.

Policy Content Hiding (PCH). PCH requires that public-chain artifacts should not reveal the semantic content of the hidden consent policy, such as data-use terms, disease categories, research purposes, institutional restrictions, or ethics-related requirements.

Policy Indistinguishability (PI). PI requires that different candidate policies should not become distinguishable from their public verification artifacts when they are compiled into the same fixed public interface. This captures whether two policies can be told apart from commitments, proofs, public parameters, or metadata even when their direct representations are hidden, that is, whether an observer can tell which policy governs an execution.

Structure and Parameter Privacy (SPP). SPP requires that public-chain artifacts should not reveal structural or parametric properties of the hidden policy, such as logical structure, nesting depth, predicate count, or set size.

4.4. Problem Statement

Given a provider-defined consent policy P, requester authorization evidence, a globally observable public blockchain, and the TPS trust assumption of Section 4.1, the prob-

lem is to enforce access to a protected healthcare resource subject to four requirements. First, authorization must be publicly verifiable and bound: the ACM, and hence any observer, can check that the requester satisfies the committed policy without learning or executing it in plaintext, and an accepting proof is tied to the complete committed policy and to the authorization context of the request (requester pseudonym, resource, and session), so that it cannot be reused for another context. Second, the policy must remain confidential: public-chain artifacts should not reveal its content or its structure and parameters, nor let it be told apart from similar policies (PCH, SPP, and PI; Section 4.3), except for terms whose enforcement the design deliberately makes public (Section 5); requesters should learn nothing about it beyond their own authorization outcomes; and requester evidence should not appear on-chain. Third, authorization decisions should be auditable: each processed request, granted or denied, is recorded on-chain together with its bound context and can be checked later without exposing the policy logic. Fourth, one policy-independent verification interface should serve every policy in the supported class, so that policies can be added or changed without changing the on-chain verifier. The first, third, and fourth requirements are met by construction (Section 5); the second is evaluated empirically against the passive public-chain observer (Section 6).

5. The Proposed Approach: TPS-ZKPI

5.1. Overview

As shown in Figure 1, the TPS-ZKPI workflow binds a provider-committed hidden policy to requester-submitted authorization evidence through TPS-side proof generation and on-chain proof verification. The provider registers only a policy commitment on-chain and sends policy-bound proving material to the TPS off-chain (steps 1–2 in the figure). For each access request, the TPS constructs the private witness from the hidden policy material and requester evidence, generates a zero-knowledge proof of policy compliance, and submits a fixed-format proof statement for ACM verification (steps 3–4). The ACM records the authorization outcome in contract storage and emits it in the blockchain event log (as access-granted or access-denied events; step 5); the data access layer consults the recorded decision to serve or refuse the request (steps 6–9).

This design separates confidentiality-sensitive proof generation from public verification. Policy material and requester evidence remain off-chain (policy material within the provider–TPS boundary, requester evidence between the requester and the TPS), while the blockchain verifies only a fixed-format proof statement against the provider's policy commitment, alongside session parameters validated by the ACM. In a traditional requester-as-prover design, the requester generates the proof itself and must therefore receive the encoded policy and the commitment-opening randomness, so every requester learns the policy in normal operation. In TPS-ZKPI, this material never leaves the provider–TPS boundary: the requester receives no policy material, and the policy itself is disclosed only if the TPS violates its trust assumption; what the public-chain transcript reveals about it is evaluated in Section 6.

5.2. Policy Representation and Interpretation

TPS-ZKPI represents heterogeneous Boolean consent policies in a fixed-length, circuit-friendly format. This policy class is chosen for two reasons. First, the supported biomedical consent conditions (DUO-style data-use terms, disease-scope constraints, and requester restrictions) are categorical conditions combined with AND/OR, and conditions that are not natively Boolean enter through fixed external-predicate slots. The generic interpreter also evaluates NOT, but the DUO compiler does not expose negation because its membership witnesses establish inclusion rather than non-membership. Second, a

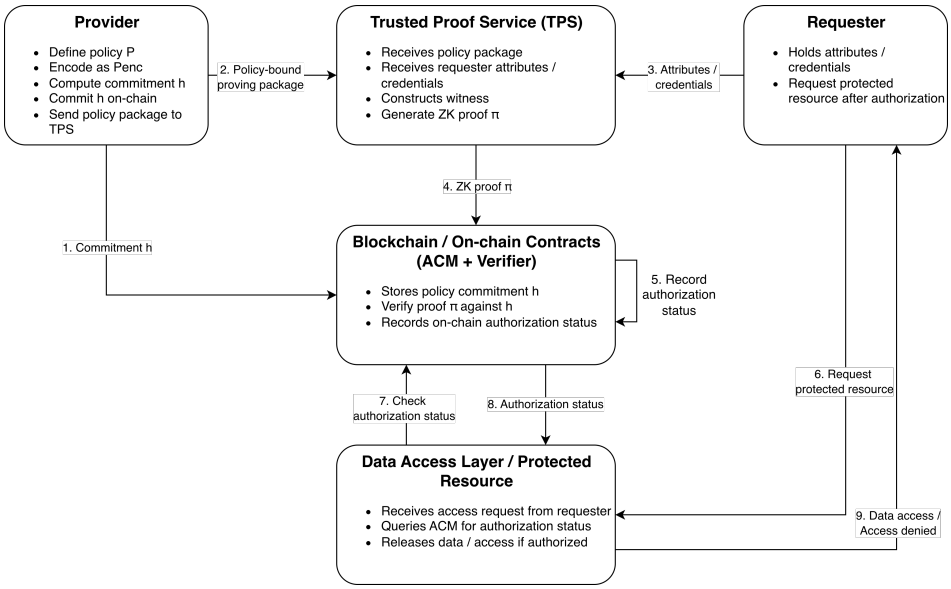


Figure 1. An overview of the TPS-ZKPI workflow.

Boolean policy class admits a fixed-length encoding evaluated by a single reusable interpreter circuit, which keeps the verification interface policy-independent, a prerequisite for the privacy objectives evaluated later. Let

$$X = \{0,1\}^N, \qquad B = \{0,1\}^M \tag{2}$$

denote the requester-attribute assignment space and the external-predicate assignment space, respectively. The full assignment space is

$$O = X \times B, \tag{3}$$

where $a \in X$ encodes requester attributes and $b \in B$ encodes predicate outcomes, including credential, ontology-membership, requester-allowlist, and obligation-related checks. The implemented relation derives these bits from their associated witnesses rather than accepting unconstrained predicate assertions.

Ontology-linked conditions are represented extensionally rather than through in-circuit ontology reasoning. At encoding time, the provider expands an ontology concept into a whitelist of codes and commits the whitelist as a Merkle root among the policy's per-predicate parameters; in-circuit, the predicate check is a set-membership proof of the requested term against this root. A disease-specific policy scoped to diabetes, for example, whitelists the diabetes subtree of the International Classification of Diseases, 10th Revision (ICD-10), whereas one scoped to type-1 diabetes whitelists only the type-1 codes. Resolving subsumption at encoding time keeps ontology reasoning out of the circuit and preserves the policy-independent interface.

Consent policies are expressed as Boolean formulas over assignment bits:

$$P ::= atom \mid (P \wedge P) \mid (P \vee P) \mid \neg P, \qquad atom ::= x_i \mid y_j. \tag{4}$$

Here x_i selects requester-attribute bit a_i and y_j selects external-predicate bit b_j. For example, the consent condition "the resource may be used for disease-specific research on diabetes, or for clinical care under approved clinical terms" is expressed as $(x_1 \wedge y_1) \vee (x_2 \wedge y_2)$, where x_1 and x_2 flag the disease-specific-research and clinical-care purposes among the requester attributes, y_1 attests that the requested disease code lies in the committed

diabetes whitelist, and y_2 attests that the requested clinical term lies in the approved clinical whitelist. Each policy is compiled into a Reverse Polish Notation (RPN) program over operand tokens and Boolean operator tokens. RPN linearizes arbitrarily nested formulas into a parenthesis-free token sequence that a stack machine evaluates in a single left-to-right pass with a fixed step count, a control structure that maps directly onto a fixed-size circuit and lets one interpreter serve all policies. To support reusable verification, each program is padded to a fixed token length L:

$$P_{\text{enc}} = [t_1, \ldots, t_L], \tag{5}$$

together with an explicit program length $\ell \leq L$ carried as a private witness. The interpreter evaluates only the first ℓ tokens; padding positions beyond ℓ are inactive and leave the output unchanged. The example policy above compiles to the token sequence $[x_1, y_1, \wedge, x_2, y_2, \wedge, \vee]$ with $\ell = 7$; the remaining $L - 7$ positions are padding. The implemented circuit uses $N = 13$, $M = 11$, and $L = \mathsf{DEPTH} = 64$. We denote the set of length-L programs by Prog_L.

The reusable policy interpreter is a zero-knowledge circuit

$$C : \mathsf{Prog}_L \times \{0, \ldots, L\} \times O \to \{0, 1\}. \tag{6}$$

Given an encoded policy, its program length, and operand values, the interpreter evaluates the RPN program using deterministic stack semantics and enforces well-formedness constraints inside the circuit. The circuit outputs 1 only if the requester evidence satisfies the encoded policy. Well-formedness is enforced as hard constraints: for a program that violates the stack discipline, no satisfying witness exists and no proof can be generated. An output of 0 therefore indicates unsatisfied evidence for a well-formed program. The proved acceptance relation is the asserted case $C(P_{\text{enc}}, \ell, (a, b)) = 1$.

The fixed-length interpreter thus realizes the policy-independent interface motivated above, and supports policy evolution through different fixed-length policy programs.

Scope of the supported policies. The policy class is restricted, and Table 4 states what the prototype establishes end-to-end. Unrestricted use (NRES) is supported only as a standalone policy; other supported terms may be combined by AND/OR in programs of at most L tokens, with each term appearing at most once and MOR/TS restricted to conjunctions. Categorical data-use terms, ontology-scoped conditions, requester allowlists, profile-backed terms, the policy validity window, and time-limited use are decided at authorization time, either inside the proved relation or by the ACM from public state; profile-backed terms establish possession of issuer-signed claims, not conformity of subsequent data use with a declared purpose. Because the ACM enforces time-limited use from public state, a policy containing TS is not content-private for that term: the commit transaction emits the duration, and the first grant emits the requester's window. The evaluated corpus contains no TS policies (Section 6.3). A publication moratorium is bound into the policy as an accepted obligation but cannot be decided at request time, and institutional or ethics approval, downstream purpose limitation, post-access duties, delegation, and credential revocation are left to external governance and the issuer lifecycle. On-chain policy revocation and resource rebinding affect future authorization decisions only; they do not retract data already released or enforce cessation or deletion after access. These lifecycle transactions are public: they reveal when a policy is replaced or revoked, but not its content. Accordingly, the prototype is an access-time consent checker for bounded Boolean policies over standardized terms.

Table 4. Consent-policy features and how far the prototype establishes them.

Consent feature	Enforcement	What is established at authorization time
Unrestricted use (NRES)	In-circuit	Compiled as a standalone unconditional policy; it cannot be combined with another DUO term.
Categorical data-use terms combined with AND/OR	In-circuit	The committed program and its length are evaluated by the interpreter; each term may appear at most once.
Disease, study-design, ancestry, geography, and clinical-term scopes (DS, RS, POA, GS, CC)	In-circuit	Merkle membership of the requested code in a provider-committed whitelist whose root is bound in the parameter commitment.
User-, project-, and institution-specific restrictions (US, PS, IS)	In-circuit	Membership of the holder pseudonym in a provider-committed allowlist for the scope.
Profile-backed terms (GRU, HMB, NMDS, GSO, NPUNCU, NPOA, NPU, NCU; Section 5.4)	In-circuit	EdDSA-Poseidon credential over policy commitment, pseudonym, resource, claims, validity interval, and identifier; the attributes are derived from the signed claims, but subsequent conformity with a declared purpose is not established.
Policy validity window	In-circuit + on-chain	The committed window is checked against the public request epoch, which the ACM requires to equal the current hour.
Time-limited use (TS)	On-chain	The committed duration is also stored and emitted in clear by the ACM; it starts a per-requester window on the first grant, and later requests outside it are denied. TS may appear only conjunctively.
Publication moratorium (MOR)	Committed	The deadline is bound into the policy as an accepted obligation; publication conduct is not observable at request time. MOR may appear only conjunctively.
Prospective consent withdrawal and policy evolution	On-chain	A successor policy supersedes its predecessor, a policy can be revoked, and a resource can be rebound for future decisions; this does not retract data already released or enforce downstream cessation or deletion.
Institutional or ethics approval (IRB)	Not provided	Documentary approval and committee review remain an out-of-circuit authorization step.
Downstream purpose limitation, post-access duties (PUB, COL, RTN), delegation, credential revocation	Not provided	Carried by external governance, the data access agreement, and the issuer lifecycle, outside the proved statement.

"In-circuit" means the condition is part of the proved relation; "on-chain" means the ACM enforces it from public state; "committed" means the value is bound into the policy but compliance is not decided at request time.

5.3. Commitment Binding and Proof Generation

To bind each proof to a provider-defined hidden policy, the provider samples commitment randomness r uniformly from the scalar field of BN254, the pairing-friendly elliptic curve over which the proofs are computed, and computes

$$\begin{aligned} q &= \mathsf{H}_{\mathrm{pol}}(P_{\mathrm{enc}}), \\ h &= \mathsf{Com}(P_{\mathrm{enc}}, \ell; r) = \mathsf{Poseidon}(q \parallel \ell \parallel r), \end{aligned} \tag{7}$$

where q is the Poseidon-based digest of the fixed-length encoded policy, ℓ is the private program length, r is the commitment randomness, and h is the policy commitment stored on-chain. The semantics of an encoded policy are determined by the pair (P_{enc}, ℓ), since ℓ identifies the active prefix of the padded token array. Accordingly, the circuit supplies the same private value ℓ to both the commitment verifier and the policy interpreter. Assuming Poseidon is collision-resistant, binding the proof to h prevents the prover from changing the committed program length. This prevents the prover from truncating P_{enc} to a shorter prefix that satisfies the stack constraints but skips part of the policy evaluation. This argument relies on the fixed-length encoding of P_{enc}, which keeps the sponge input unambiguous. Hiding is an explicit commitment assumption (Section 4.1) that relies on the high-entropy randomness r.

For an access request, let

$$\chi = (id_{\text{req}}, id_{\text{res}}, \nu), \qquad d_\chi = \mathsf{Poseidon}(id_{\text{req}} \parallel id_{\text{res}} \parallel \nu), \tag{8}$$

where ν is a fresh session nonce. Here id_{req} is the requester pseudonym. For policies that activate a holder-bound credential or requester-allowlist predicate, the circuit derives it as $\mathsf{Poseidon}(s_{\text{holder}})$ from a private holder secret s_{holder}; otherwise, it may be a requester-supplied pseudonym. The TPS constructs the private circuit witness

$$w = (P_{\text{enc}}, \ell, a, b, r, \omega, \chi), \tag{9}$$

where P_{enc} is the encoded policy, ℓ is its program length, a denotes requester attributes, b denotes external predicate outcomes, r is the commitment randomness, and ω denotes auxiliary witnesses for credential- or predicate-dependent checks. The context tuple χ consists of private circuit inputs, while its digest d_χ is public. The current public-signal vector is

$$(1, h, pp, d_\chi, \tau), \tag{10}$$

where pp is a fixed-format commitment to the per-predicate parameters and τ is the request epoch used by the temporal checks. Although the three components of χ are not separate Groth16 public inputs, the ACM receives them as transaction parameters in order to recompute d_χ; they are therefore visible in transaction calldata.

For the compiled circuit, the Groth16 [21] setup produces the common reference string as a proving key and a verification key, once for the shared circuit:

$$(pk, vk) \leftarrow \mathsf{Setup}(C). \tag{11}$$

The TPS generates a proof

$$\pi \leftarrow \mathsf{Prove}\big(pk, w; (1, h, pp, d_\chi, \tau)\big). \tag{12}$$

The proof certifies the conjunction that $C(P_{\text{enc}}, \ell, (a, b)) = 1$, $h = \mathsf{Com}(P_{\text{enc}}, \ell; r)$, $d_\chi = \mathsf{Poseidon}(\chi)$, and the selected external-predicate witnesses satisfy their policy-bound verification relations. Before verifying π, the ACM checks that the resource is governed by the stated policy, that τ is current, that the nonce has not been used for the same requester–resource context, and that the digest recomputed from the transaction parameters equals the public signal d_χ. Thus, within one ACM deployment, a proof generated for one requester pseudonym, resource, and nonce cannot be accepted under a different tuple. The ACM then verifies π against vk and records the outcome of each request: a positive authorization outcome requires both a valid proof and an asserted output of 1; otherwise the request is recorded as a denial. The policy encoding, commitment randomness, requester attributes, credential witnesses, and auxiliary proof material are not posted on-chain.

5.4. Credential-Origin Binding

The circuit reserves one fixed predicate slot for a credential carrying a fixed set of DUO-relevant claims. Let $c_0, \ldots, c_6$ be the private, issuer-signed claim bits for, respectively, research use, health or medical use, methods development, genetic study, ancestry, commercial use, and non-profit use. Let t_{from} and t_{until} define its validity interval, and let γ be its credential identifier. Let d_{DUO} be the fixed credential schema/domain tag for this credential type. From these signed claims, the circuit derives the DUO terms GRU, HMB, no general methods research (NMDS), genetic studies only (GSO), not-for-profit, non-commercial use only (NPUNCU), population origins or ancestry research prohibited

(NPOA), not-for-profit use only (NPU), and non-commercial use only (NCU). For issuer public key $A = (A_x, A_y)$, the circuit reconstructs the domain-separated message

$$m_{\text{cred}} = \mathsf{Poseidon}(d_{\text{DUO}} \parallel h \parallel id_{\text{req}} \parallel id_{\text{res}} \parallel c_0 \parallel \cdots \parallel c_6 \parallel t_{\text{from}} \parallel t_{\text{until}} \parallel \gamma) \tag{13}$$

Before proof generation, an issuer holding the corresponding private signing key s_{iss} issues the credential by computing

$$\sigma \leftarrow \mathsf{Sign}_{\text{EdDSA-Poseidon}}(s_{\text{iss}}, m_{\text{cred}}). \tag{14}$$

The resulting credential is supplied to the TPS as private off-chain evidence for proof generation. At proof generation, the TPS supplies A, σ, and the credential fields as private witness values. The circuit then enforces

$$\mathsf{Verify}_{\text{EdDSA-Poseidon}}(A, \sigma, m_{\text{cred}}) = 1, \qquad \mathsf{Poseidon}(A_x \parallel A_y) = k_{\text{iss}}, \tag{15}$$

where k_{iss} is the issuer-key commitment contained in the policy-bound parameter commitment pp. The circuit additionally requires $t_{\text{from}} \leq \tau \leq t_{\text{until}}$ and $\gamma \neq 0$. Because this credential is holder-bound, a policy that activates this slot also requires $id_{\text{req}} = \mathsf{Poseidon}(s_{\text{holder}})$, binding the credential to the requester pseudonym in the authorization context. The corresponding DUO attributes are constrained to these claim-derived values.

5.5. Transcript Normalization

Zero-knowledge proofs hide private witness values, but surrounding blockchain artifacts may still leak policy-dependent information. TPS-ZKPI therefore normalizes the public-chain transcript so that different policies induce fixed or tightly bounded observable profiles. Table 5 summarizes the mechanisms.

Table 5. Transcript-normalization mechanisms in TPS-ZKPI.

Artifact	Normalization mechanism
Calldata	Fixed verification interface and fixed-size proof/public-input encoding.
Logs	Fixed event structure with constant topic and payload layout.
Public parameters	Fixed-format handles instead of policy artifacts or credential witnesses.
Gas usage	Policy-independent verifier logic and gas padding that counts calldata byte costs and tops each transaction up to a target drawn at random from a fixed band.

These mechanisms reduce policy-dependent variation in calldata, logs, public parameters, and gas usage that could otherwise serve as a fingerprint for the passive public-chain observer of Section 4.

6. Experiments and Results

We evaluate PCH, PI, and SPP (Section 4.3) against a passive public-chain observer (Section 6.1). The evidence comes from two attack families run on the same executions: a supervised attacker that tries to recover each hidden label from the on-chain observables, and an unsupervised attacker that asks whether the observables cluster along that label without seeing it. Across the 45 tasks, two meet the leakage criterion (one ICD-10 chapter and one nesting-depth contrast); both trace to the randomly drawn padding target, and the unsupervised attacker finds no natural grouping by the hidden labels beyond the same padding effect. Table 6 summarizes the headline supervised verdicts across the three privacy objectives. Beyond the privacy verdicts, we report the system's performance as a cost profile: per-transaction gas for commit and access (including the padding overhead), calldata sizes, circuit constraint count, and proving time, together with how circuit capacity, fixed-circuit policy complexity, and offline proof throughput scale.

Table 6. Supervised attack results on the Polygon Amoy dataset.

Objective	Hidden target	Results
PCH	DUO terms and ICD-10 chapters	AUC 0.464–0.570, significant seeds median 3%
PI	Which policy of a related pair is executed	AUC 0.509–0.549; significant seeds median 7%
SPP	Structure, predicate, parameter, and nesting-depth buckets	AUC 0.468–0.571, significant seeds median 7%

6.1. Adversary and Observables

We instantiate the passive public-chain observer of Section 4: an adversary who reads the ledger without restriction and sees every transaction and event our approach emits, but never the plaintext policy or the witness.

Under the assumptions of Section 4.1, the byte values in the adversary's public view carry no policy information: the proof is zero-knowledge, the commitments in the public statement are hiding, and the authorization output is identical for every execution in the corpus.

The attack therefore targets the metadata of these artifacts: for each policy execution (a commit transaction followed by an access transaction) it extracts the following features:

- **Gas:** `commitGasUsed`, `accessGasUsed`.
- **Calldata length:** `commitCalldataLength`, `accessCalldataLength`.
- **Proof size:** `proofSize`.
- **Event logs:** `commitLogsCount`, `commitLogsSize`, `accessLogsCount`, `accessLogsSize`.

Of these features, calldata length, proof size, and log counts/sizes are constant across the evaluated policies by design; any policy-dependent signal can therefore only flow through the two gas measurements (`commitGasUsed`, `accessGasUsed`). The fixed circuit and the commitment-only interface make the non-gas features constant; randomized gas padding protects the gas channel that remains. That channel is wider than the verifier's arithmetic: because the Ethereum Virtual Machine (EVM) prices calldata per byte (four gas for a zero byte, sixteen otherwise), the gas of an access transaction also encodes the byte pattern of its public arguments, so any deterministic difference in how an argument is encoded reaches the observer through gas even when the calldata length is constant (Section 6.9 exhibits one such difference). The constant features remain in the attack pipeline but carry no variation to exploit. Time-limited use (TS) is the exception by design: the ACM enforces it from public state, so a TS policy discloses the term and its duration through additional events (Section 5); the evaluated corpus contains no TS policies. The evaluation is scoped to these observables; block timing, transaction ordering, mempool or broadcast patterns, off-chain TPS communication, adaptive or repeated requester queries, and correlations between authorization outcomes and policy classes (every evaluated request is granted) lie outside this scope, as does a TPS that deviates from its workflow (Section 4.1).

We consider three adversary goals, one per privacy objective: (i) recover the policy's *content* (PCH, Section 6.4); (ii) distinguish two *semantically related* policies (PI, Section 6.5); and (iii) recover the policy's *structure or parameter sizes* (SPP, Section 6.6).

6.2. Attack Methodology

We evaluate each privacy objective with a primary supervised attack and a complementary unsupervised cross-check. The *supervised* method asks whether a trained classifier can recover a hidden label from the observables, measured by the area under the

receiver operating characteristic curve (AUC) against the chance level of 0.5. Its outcome serves as the main privacy verdict. The supervised adversary has strictly more information than the unsupervised one, and is thus the stronger of the two. Because this adversary is harder to defeat, a PASS under the supervised test is the more conservative verdict. The *unsupervised* method asks whether the observables cluster along the hidden label, measured by the Adjusted Rand Index (ARI) and Normalized Mutual Information (NMI) against their chance level of 0. This chance level is exact for ARI but only nominal for NMI, which is positively biased at small samples (Section 6.8).

Execution setup. All policy executions are transactions on the *Polygon Amoy* testnet: for each generated policy, we commit the encoded policy on-chain from a provider account (`commitPolicyWithGasPad`), generate the Groth16 proof locally in the role of TPS (Section 4.1), and submit the access transaction from a separate TPS account (`requestAccessBoundWithGasPad`). The provider-to-TPS and requester-to-TPS exchanges are in-process calls: they use no public channel and produce no on-chain artifacts. They therefore do not affect the observables of Section 6.1, which are extracted from the resulting transaction receipts, calldata, and event logs.

Both methods run on the same dataset of *Polygon Amoy* executions, collected under the full defense configuration (fixed circuit, commitment-only interface, randomized gas padding). The dataset, attack code, and verdict files are released as an accompanying artifact (see the Data Availability Statement).

6.2.1. Supervised Attacks

Each objective is operationalized as a set of binary *tasks*, each defined by one hidden label to be recovered from the observables: detecting the presence of a specific data-use term or disease chapter (PCH, Section 6.4), distinguishing executions of two policies (PI, Section 6.5), or separating two structural buckets (SPP, Section 6.6). For every task we train a linear and a non-linear classifier (logistic regression, LR; random forest, RF) and report the score that enters the verdict for that objective (defined below). Each task is evaluated over **30 random seeds** using a **policy-disjoint train/test split** (the same 70/30 ratio in each of PCH, PI, and SPP): the policies used to train the attacker never appear in the test set, so a high test AUC reflects leakage that generalizes across policies. Because the split assigns whole policies rather than individual executions, we cannot partition executions to hit the target class proportions exactly. For each seed we therefore approximate stratification by sampling many candidate splits (512 per seed in our runs) and keeping the one whose per-class test proportions are closest to 70/30. Here a class is one side of the binary task (the positive or negative value of its hidden label). By keeping each class's test share close to the target ratio, the candidate search ensures every seed's test set contains both classes, so each seed's AUC is estimated on a two-class test set.

For each seed we assess the AUC with a label-permutation test. The classifier is trained once and its test-set probability outputs are fixed; the test-set labels are then shuffled R=500 times, and the AUC is recomputed against the fixed probabilities on each shuffle, giving the null distribution of the AUC under no association between scores and labels. The empirical p-value is

$$p = (k+1)/(R+1), \tag{16}$$

where k is the number of shuffles whose AUC meets or exceeds the observed (unshuffled) AUC; the added counts keep the p-value away from an exact zero. The seed is counted as significant when this p-value falls below α=0.05.

For each task, the result tables report the AUC averaged over seeds (AUC_{mean}) and the AUC gap, the mean per-seed distance from chance:

$$\text{AUC gap} = \text{mean}_s|\text{AUC}_s - 0.5| \;\geq\; |\text{AUC}_{\text{mean}} - 0.5|. \tag{17}$$

Because the gap is computed before averaging, deviations above and below 0.5 cannot cancel, which is why it is never smaller than the distance of AUC_{mean} from chance.

Because every task is binary, its AUC has a chance level of 0.5 regardless of class balance. Thus, differences in positive and negative sample counts across the generated corpus do not shift the chance baseline, although they may affect estimation uncertainty.

The PASS/FAIL decision, defined below, combines AUC_{mean} with the significant-seed fraction (the fraction of seeds whose per-seed permutation test rejects at α=0.05), reported as %sig in the result tables. The verdict is:

> **FAIL** iff $AUC_{mean} > 0.55$ **and** the significant-seed fraction is ≥ 0.5 (over 30 policy-disjoint seeds); **PASS** otherwise.

We refer to this rule as the *leakage criterion*. Both of its conditions are required to trigger a FAIL verdict: the AUC threshold imposes a minimum effect size, while the significant-seed threshold requires the effect to recur across policy-disjoint splits. A task that does not satisfy both conditions is labeled PASS; PASS does not constitute proof of zero leakage.

Operational thresholds and sensitivity. The two conditions guard against different errors. The AUC threshold is an effect-size floor of 0.05 above chance: with large test sets, a permutation test can reject for a negligible AUC excess, and the floor keeps such an effect from triggering a FAIL. The stability condition requires the permutation-test p-value to be below 0.05 in at least 15 of the 30 policy-disjoint splits. Under the null hypothesis, each split rejects with probability 0.05, so a majority of rejections indicates an effect that recurs across splits rather than one carried by a few favorable splits. Because the splits overlap, the significant-seed fraction is a stability measure rather than a combined p-value. In the evaluated run, the stability condition is the binding one: the AUC condition alone would flag four PCH and four SPP tasks, three of each being significant in fewer than half of the splits, whereas the stability condition alone flags the same single task per experiment as the full leakage criterion. The AUC floor therefore acts as a safeguard rather than as the deciding condition.

Because these thresholds are operational choices, we reapply the criterion over AUC thresholds of 0.52–0.60 and significant-seed fractions of 0.3–0.7. At fractions of at least 0.5, at most the two tasks discussed in the PCH and SPP result sections fail; lowering the fraction to 0.3–0.4 adds at most two SPP contrasts, whereas raising it to 0.6 yields none. No PI task fails under any of these threshold pairs. In ten null reruns with policy-independent gas features, four contain a spurious PCH failure (a 3% per-task frequency), while none produces an SPP failure. We therefore treat FAIL as a diagnostic flag and examine the two observed failures in the corresponding result sections without changing their verdicts.

The effectiveness of gas padding is assessed separately by the paired gas-padding ablation (Section 6.9). Using the same policies, features, splits, and classifiers, the attacks recover signal from the unpadded transcript, and that signal collapses after padding. Thus, the leakage criterion summarizes residual attack performance, whereas the paired reduction provides the primary empirical evidence that padding suppresses the modeled gas channel.

Tasks with a median test-set size below 20 are reported as *underpowered* and excluded from the verdict. The criterion is applied independently to LR and RF; a task fails if either classifier satisfies both conditions, and the result tables report the classifier with the higher mean AUC.

The classifiers receive all nine features of Section 6.1; only the commit and access gas measurements vary, so they carry the only modeled policy-dependent channel that remains after transcript normalization.

6.2.2. Unsupervised Clustering Attacks

The clustering attacker drops the labels entirely and asks whether the gas features *naturally* group along the hidden property. We run K-Means (with K set to the number of classes, or K=2 for policy pairs) on the on-chain gas features, and score the recovered clusters against the true labels with ARI and NMI. Values near 0 mean the clusters carry no information about the hidden label; values near 1 mean full leakage.

PCH clusters all executions globally, and SPP is clustered the same way on its 1,125-execution view (Section 6.3), once per annotation axis of Section 6.6 with K equal to that axis's bucket count. At these sample sizes, ARI and NMI are read directly, each averaged over 20 K-Means seeds. To test whether an ARI exceeds chance, the cluster assignments are kept fixed, the labels are shuffled 200 times, and p_{ARI} follows Equation (16) with R=200, so its smallest attainable value is $1/201 \approx 0.005$.

For PI, clustering is scored per pair, so each side of a pair must contribute enough executions; this requires two safeguards. **Balanced pairs.** The clustering uses a dedicated pair set built from the same executions: the policies with ≥ 30 executions each (the depth layer of Section 6.3) are paired into 127 balanced 30-vs-30 contrasts. The pair set of the supervised PI attack (Section 6.5) is unsuitable for per-pair clustering: most of its pairs hold a single execution per side, and its most imbalanced hold 1 execution against 30, so K-Means cannot recover a meaningful partition, and ARI/NMI ≈ 0 would reflect the degenerate setup rather than privacy. (The supervised attack itself is unaffected: it pools executions within a pair type rather than partitioning each pair.) **Permutation-debiased NMI.** Even after balancing, each side of a pair holds only 30 executions. At that scale the null simulation in Section 6.8 shows that finite-sample effects alone can raise NMI above zero while ARI stays at chance, so ARI is read directly and NMI must be debiased. The cluster assignments are computed once (K-Means does not see the labels) and the observed NMI is averaged over 20 K-Means seeds; the pair labels are then shuffled 200 times and the NMI recomputed against the fixed assignments on each shuffle, giving the null distribution of NMI under zero true signal. The debiased NMI is the observed NMI minus the mean of this null distribution, and the empirical p-value p_{NMI} is the fraction of shuffles whose NMI meets or exceeds the observed value. The cross-check flags a pair only if its debiased NMI is positive *and* significant ($p_{\mathrm{NMI}} < 0.05$).

6.3. Dataset and Corpus Generation

Shared corpus. We evaluate all three objectives on one policy corpus, so the supervised attacks and the clustering cross-check see the same transactions. The corpus holds 1077 distinct policies run as 1773 on-chain executions on *Polygon Amoy*, each execution being one commit transaction followed by one access transaction. The policies are produced by a deterministic generator with a fixed seed (`unified-decoupled-2026`). For reproducibility, this generator uses a deterministic test issuer to create the private profile-credential witnesses used by policies that exercise the credential predicate; this testing key does not model issuer governance or production key management.

For each policy the generator picks its content and its structure. The content is a set of DUO data-use terms drawn from ten terms: five ontology-bound terms, each checked against a code set from a biomedical vocabulary (DS against ICD-10, RS (research-specific restrictions) against the Ontology for Biomedical Investigations, OBI [40], POA against the Human Ancestry Ontology (HANCESTRO), GS (geographical restriction) against the

ISO 3166 country codes, CC against SNOMED Clinical Terms (SNOMED CT)), and five profile-backed terms (GRU, HMB, NMDS, GSO, NCU) that are derived from the signed claims of the requester's profile credential (Section 5). The structure is the logical form joining the terms; whitelist sizes are drawn once per policy from a fixed set of candidate sizes, independently of the logical form, so that the four structural axes of Section 6.6 are not functions of one another. For example, one generated policy permits research-specific and disease-specific use (DUO terms RS and DS) and grants access only when *both* conditions hold, joined by `AND`: the study design belongs to the OBI *observational and comparative designs* family, and the requested record falls in the ICD-10 *diseases of the circulatory system* chapter. Each condition is one predicate (hence the two predicates), and each is checked against a whitelist of the codes it admits: 8 OBI study-design codes and 8 ICD-10 codes drawn from that chapter, the "medium" parameter-size bucket of Table 7.

Table 7. Composition of the shared policy corpus

Grouping	Category	Policies	Executions
DUO data-use term (multi-label)			
	GRU	302	447
	DS	422	770
	RS	412	760
	POA	392	624
	GS	382	556
	CC	392	769
	NCU	206	380
	HMB	331	447
	NMDS	237	469
	GSO	236	352
ICD-10 chapter (policies with an ICD-10 scope, i.e., DS policies)			
	ch0	44	131
	ch1	49	49
	ch2	33	62
	ch3	55	55
	ch4	41	70
	ch5	34	63
	ch6	48	135
	ch7	48	77
	ch8	33	62
	ch9	37	66
	Subtotal	422	770
SPP structureType			
	trivial	130	130
	and_only	294	526
	or_only	303	535
	and_or_mixed	350	582
SPP nestingDepth			
	flat	130	130
	shallow	210	413
	medium	335	509
	deep	402	721
SPP predicateCombo			
	single	257	344
	few	645	1080
	many	175	349
SPP paramType			
	none	147	234
	small	232	406
	medium	352	584
	large	346	549
Corpus total		**1077**	**1773**

ICD-10 labels ch0–ch9 index the ten chapters used: A00–B99, C00–D48, D50–D89, E00–E90, F01–F99, G00–G99, I00–I99, J00–J99, K00–K93, and M00–M99.

Every code set is a subset of its vocabulary, and the recorded whitelist size is the number of leaves actually committed. Table 7 gives the resulting counts for every label and structural bucket.

Two-layer execution design. We run the policies in two layers that serve different needs. The *coverage layer* runs 1053 policies once each. It provides breadth, spreading distinct policies across the content labels and structural buckets, so the PCH and SPP attacks have enough different policies to train on one set and test on another.

The *depth layer* runs 24 policies 30 times each (720 executions). It provides depth, which the PI attacks need because they compare two policies execution-by-execution, and for that each policy must have multiple executions of its own. This layer is what lets PI form its 127 balanced 30-vs-30 pairs and run the per-pair clustering check (Section 6.2). The PCH and PI analyses use all 1,773 executions; the SPP analyses, supervised and unsupervised, keep the first three executions of each depth-layer policy, so they use 1,125 executions, to which the 24 repeated policies contribute 72 rather than 720.

Label restriction and statistical power. We restrict the generated content labels to those with enough support for a statistically powered attack: ten DUO data-use terms and ten ICD-10 chapters (Table 7), sampled so that the smallest class reaches at least 30 executions. The complete DUO has 23 data-use terms and ICD-10 has tens of thousands of leaf codes; generating and executing enough policies for every leaf term to reach adequate power would make the on-chain experiment infeasible. Category-level labels are therefore the finest granularity at which the attack has adequate power. They do not directly test individual leaf terms: a signal specific to one term could be diluted within its category, so a PASS at the category level does not rule out a term-specific leak. The transcript mechanisms themselves do not depend on the policy content: the verifier work and the public interface are fixed, and the padding band is the same for every policy (Section 5).

Collection. Executions were collected as live transactions on *Polygon Amoy* under the full defense configuration of Section 6.2 (fixed circuit, commitment-only interface, randomized gas padding); observables are extracted from the resulting receipts, calldata, and event logs. All executions completed with the outcome the generator planned, and the feature file holds all of them.

6.4. Policy Content Hiding (PCH)

This attack targets the DUO data-use terms of the corpus, namely general research use (GRU), disease-specific research (DS), research-specific restrictions (RS), population origins or ancestry research (POA), geographical restriction (GS), clinical care use (CC), non-commercial use (NCU), health or medical or biomedical research (HMB), no general methods research (NMDS), and genetic studies only (GSO), as well as disease codes aggregated to ten ICD-10 chapters (Table 7).

Each DUO term defines a one-vs-rest binary task, on which we run the supervised pipeline stated in Section 6.2. Table 8 reports the per-term verdicts: every DUO task stays below the 0.55 AUC line, with mean AUC between 0.477 and 0.540; significant-seed fractions run up to 23% (GRU), short of the 50% threshold, so no DUO task meets either condition of the leakage criterion.

The same pipeline applied to the ten ICD-10 chapters (one-vs-rest over the DS policies only, which carry the ICD-10 scope; median test n=127 per chapter; Table 9) passes nine chapters and fails one. Chapters ch1, ch2, and ch4 sit just above the AUC line (mean AUC 0.558–0.570) but reproduce in at most 23% of seeds, so they are boundary cases. Chapter ch5 (diseases of the nervous system) meets both conditions of the criterion: mean AUC 0.667 with 53% of seeds significant. We report it as a FAIL of the criterion and examine below what the classifier separates.

Table 8. Policy Content Hiding: per-term supervised attack over 30 policy-disjoint seeds (one-vs-rest tasks; higher-mean-AUC classifier per task; med n: median test-set size).

Term	AUC (mean ± std)	AUC gap (mean)	%sig	med n
DUO GRU	0.540 ± 0.026	0.041	23%	323
DUO DS	0.514 ± 0.029	0.027	3%	323
DUO RS	0.498 ± 0.028	0.022	0%	323
DUO POA	0.477 ± 0.024	0.027	0%	323
DUO GS	0.502 ± 0.027	0.023	0%	323
DUO CC	0.515 ± 0.029	0.025	3%	323
DUO NCU	0.528 ± 0.027	0.031	7%	323
DUO HMB	0.509 ± 0.027	0.024	3%	323
DUO NMDS	0.516 ± 0.030	0.025	3%	323
DUO GSO	0.502 ± 0.031	0.026	3%	323

Table 9. Policy Content Hiding: per-chapter ICD-10 supervised attack over 30 policy-disjoint seeds (higher-mean-AUC classifier per chapter; med n: median test-set size).

Chapter	AUC (mean ± std)	AUC gap (mean)	%sig	med n
ICD ch0	0.472 ± 0.078	0.064	0%	127
ICD ch1	0.570 ± 0.066	0.075	23%	127
ICD ch2	0.565 ± 0.071	0.081	10%	127
ICD ch3	0.540 ± 0.061	0.061	7%	127
ICD ch4	0.558 ± 0.061	0.070	13%	127
ICD ch5	0.667 ± 0.069	0.167	53%	127
ICD ch6	0.464 ± 0.077	0.065	0%	127
ICD ch7	0.525 ± 0.064	0.058	3%	127
ICD ch8	0.478 ± 0.078	0.063	0%	127
ICD ch9	0.511 ± 0.074	0.062	0%	127

What the ch5 classifier separates. The two gas measurements are the only non-constant observables. For each padded access transaction, the recorded gas can be decomposed as

$$\texttt{accessGasUsed} = \texttt{padTarget} + \texttt{residualOverhead}, \tag{18}$$

where the TPS draws `padTarget` uniformly from the padding band (Section 5). On Polygon Amoy, the residual overhead ranges only from 4,661 to 4,750 gas across all 1,773 executions, so almost all variation in the padded access gas comes from the randomized target. The observer sees only the total gas, whereas the capture log also records each padding target. We therefore rerun the analysis on the residual overhead, that is, on the access gas minus the padding target, to determine whether the ch5 separation remains after removing the randomized component. **This subtraction is diagnostic of the padded transcript only: the resulting residual overhead is not the gas usage of an unpadded execution.**

Across all 20 PCH tasks, the single-feature rank AUC of the padded access gas, computed over all executions without a classifier, coincides with that of the padding target to three decimal places, while the residual AUC lies between 0.47 and 0.52. For ch5, the gas and target AUCs are both 0.601, whereas the residual AUC is 0.472. The 34 ch5 policies happened to draw targets whose mean is 33,000 gas higher than that of the other 388 DS policies, a 3.2σ deviation for a uniform draw of that size. Because each coverage-layer policy is executed once, the 30 evaluation seeds repeatedly partition the same fixed draws, causing this accidental difference to reproduce across seeds.

Applying the evaluation protocol to the residual gives a mean AUC of 0.556 with 10% significant seeds for logistic regression and 0.561 with 17% significant seeds for ran-

dom forest, both yielding PASS. Applying it to the padding target alone reproduces the observed FAIL (0.686, 63%). Moreover, in the separate unpadded ablation, ch5 is already near chance (AUC 0.468 for logistic regression and 0.455 for random forest). Thus, for ch5 specifically, we find no detectable policy-dependent gas signal without padding; the FAIL in the padded corpus is instead caused by an accidental finite-corpus correlation between the random padding targets and the ch5 labels. This task-specific result does not imply that padding is unnecessary: the unpadded ablation reveals substantial leakage for many other policy contrasts, as shown in Section 6.9.

6.5. Policy Indistinguishability (PI)

To evaluate PI, we pair policies using two quantities. The semantic similarity score is a fixed heuristic that returns 1 when two policies share an identical normalized logic string, that is, the same DUO terms joined by the same Boolean form; the string names the vocabulary of each condition but not its value, so two such policies may restrict, for example, different ICD-10 chapters. For all other couples, the score combines the Jaccard overlap [41] of DUO terms with agreement of the ICD-10, OBI, and other scope fields. The parameter fingerprint consists of the whitelist size, Merkle depth, and proof type.

Each candidate policy couple is assigned to one of three types: `semantic_equivalent` (similarity ≥ 0.999 and different fingerprints), `weak_equivalent` (similarity ≥ 0.95 without that fingerprint difference), and `controlled_difference` (similarity < 0.95, the least similar pairs by construction; reported for reference but excluded from the verdict). We sample a fixed budget of 100 pairs per type, or every available pair when fewer exist (71 `weak_equivalent` pairs); each retained pair is an ordered couple (P_0, P_1) that the supervised attack below tries to tell apart.

In this corpus, every couple at or above 0.999 shares the logical form, so the `semantic_equivalent` pairs differ in their parameters and possibly in their scope values. 70 of the 71 `weak_equivalent` pairs share the logical form as well, and 30 of these are content-identical policies that differ only in their commitment randomness; despite its name, this type therefore holds the closest pairs of the corpus.

Supervised attack. Each pair is an ordered couple (P_0, P_1), and the executed policy defines a binary label $b \in \{0, 1\}$; the attacker's task is to predict b from the on-chain features of a single execution, i.e. to decide which policy of the pair produced it. Indistinguishability is thus a two-class problem whose ideal outcome is a guess no better than a random guess, corresponding to a classifier AUC of 0.5.

We train a logistic-regression and a random-forest classifier and, per pair type, report the one with the higher mean AUC, following the convention of Section 6.2. We evaluate this single-execution attacker over 30 policy-disjoint seeds (Table 10). Pooled over all pairs, the AUC is 0.509 ± 0.035 with 3% of seeds significant. Both pair types that enter the verdict stay below the 0.55 line: `semantic_equivalent` at 0.549 (7% significant seeds) and `weak_equivalent` at 0.544 (10%), so PI passes. `controlled_difference`, the least similar pairs by construction and hence the easiest to separate, is excluded from the verdict and reported for reference; under the full defense configuration it sits at 0.544 with 13% of seeds significant, whereas the ablation of Section 6.9 separates the same pair type at AUC 0.68 once padding is removed (Table 14), which confirms that the pair construction produces a distinguishable positive control.

6.6. Structure and Parameter Privacy (SPP)

To evaluate SPP, we annotate each policy along four axes that instantiate the structural and parametric properties of Section 4.3, each partitioned into a few discrete buckets (categories or value ranges, listed below). The four axes are logical structure type

Table 10. Policy Indistinguishability: single-execution supervised attack over 30 policy-disjoint seeds, by policy-pair type (higher-mean-AUC classifier per type).

Pair type	Pairs	AUC (mean ± std)	AUC gap (mean)	%sig	med n
overall (all types)	271	0.509 ± 0.035	0.027	3%	162
semantic_equivalent	100	0.549 ± 0.064	0.070	7%	60
weak_equivalent	71	0.544 ± 0.079	0.079	10%	42
controlled_difference †	100	0.544 ± 0.064	0.062	13%	60

† Reported for reference; excluded from the PI verdict.

(`structureType`), predicate count (`predicateCombo`), parameter size (`paramType`), and nesting depth (`nestingDepth`). We treat each pair of buckets within an axis as one binary task: the attacker must tell from the observables which of the two buckets the executed policy falls in.

`structureType` classifies the logical form joining the DUO terms (Section 6.3) into four buckets: `trivial` (a single term with no Boolean connective, e.g. policy logic `"DS"`), `and_only`, `or_only`, and `and_or_mixed`. `predicateCombo` buckets policies by predicate-namespace count into `single` (1), `few` (2–3), and `many` ($\geq$ 4); `paramType` buckets by whitelist size into `none` (no whitelist: the policy is decided by profile-backed terms only), `small` (1–4), `medium` (5–16), and `large` ($\geq$ 17); `nestingDepth` buckets by the maximum nesting of the normalized expression, in which each term is joined by AND to the predicate that checks it (e.g. `"DS"` becomes `(DS & P[ICD10])`), into `flat` ($\leq$ 1), `shallow` (2), `medium` (3), and `deep` ($\geq$ 4). The generator draws the whitelist size independently of the logical form, so the four axes are not functions of one another; the per-bucket counts of the full corpus are listed in Table 7, and the SPP attacks use 1,125 of its executions (Section 6.3).

Supervised attack. Table 11 reports pairwise contrasts over 30 policy-disjoint seeds, with the per-contrast AUC taken from the classifier with the higher mean AUC, following the convention of Section 6.2. Most contrasts sit at chance, with mean AUC between 0.47 and 0.54. Three exceed the 0.55 line without reproducing: `paramType` `none/small` (0.571, 40% significant seeds), `none/large` (0.568, 33%), and `nestingDepth` `flat/medium` (0.555, 27%); the two `none` contrasts separate the profile-only policies, whose 147 policies form the smallest minority class of the axis. One contrast, `nestingDepth` `medium/deep`, meets both conditions of the criterion (mean AUC 0.563, 53% significant seeds) and is reported as FAIL.

Its separability has the same origin as the PCH ch5 case (Section 6.4). The single-feature rank AUC of the access gas for this contrast equals that of the padding target alone (0.440 for both, i.e. deep policies drew lower targets) and the residual after subtracting the target scores 0.467; over all 21 contrasts the gas and target AUCs agree to within 0.001, and the residual AUC stays within 0.47–0.56. Rerunning the protocol with the residual in place of the access gas returns the contrast to PASS (logistic regression 0.530 with 13% significant seeds; random forest 0.544 with 33%), while the target alone reproduces the FAIL (0.568, 57%). What remains above chance on this contrast is the per-policy random draw that the padding itself introduces, not a stable structural channel. Section 6.9 provides the corresponding positive control: when gas padding is removed, the same SPP attacker separates structural and parameter-size buckets with AUCs up to 0.90, and that signal collapses once padding is applied.

6.7. Unsupervised Clustering: A Label-Free Cross-Check

The supervised attacks (Sections 6.4–6.6) assume an adversary who can train on labeled executions. A weaker, label-free attacker asks whether the on-chain gas features *cluster* along the hidden property with no supervision at all.

Table 11. Structure and Parameter Privacy: 21 pairwise contrasts across four annotation axes over 30 policy-disjoint seeds (higher-mean-AUC classifier per contrast; med n: median test-set size).

Contrast	AUC (mean ± std)	AUC gap (mean)	%sig	med n
structureType				
trivial / or_only	0.520 ± 0.048	0.043	0%	130
trivial / and_only	0.502 ± 0.040	0.033	3%	127
trivial / and_or_mixed	0.502 ± 0.042	0.034	0%	144
or_only / and_only	0.506 ± 0.034	0.027	0%	179
or_only / and_or_mixed	0.518 ± 0.035	0.032	10%	196
and_only / and_or_mixed	0.503 ± 0.040	0.032	7%	193
predicateCombo				
single / few	0.523 ± 0.035	0.033	13%	271
single / many	0.519 ± 0.048	0.041	7%	130
few / many	0.527 ± 0.048	0.045	17%	246
paramType				
none / small	0.571 ± 0.046	0.071	40%	114
none / medium	0.525 ± 0.035	0.035	3%	150
none / large	0.568 ± 0.038	0.069	33%	148
small / medium	0.530 ± 0.037	0.037	20%	175
small / large	0.504 ± 0.036	0.028	3%	173
medium / large	0.493 ± 0.027	0.022	0%	209
nestingDepth				
flat / shallow	0.522 ± 0.038	0.036	3%	102
flat / medium	0.555 ± 0.032	0.055	27%	140
flat / deep	0.468 ± 0.046	0.047	3%	160
shallow / medium	0.521 ± 0.041	0.039	7%	164
shallow / deep	0.533 ± 0.036	0.039	17%	184
medium / deep	0.563 ± 0.028	0.063	53%	221

Table 12. Unsupervised clustering attack (K-Means, K = number of classes, K=2 for pairs) on the on-chain gas features.

Objective	Target	n	#cls.	ARI	NMI
PCH	DUO terms	1773	10	0.0000	0.0109
PCH	ICD-10 chapters	770	10	0.0021	0.0281
PI	policy pairs (30v30)	127	2	0.0020	0.0139
SPP	`structureType`	1125	4	0.0004	0.0034
SPP	`predicateCombo`	1125	3	−0.0005	0.0015
SPP	`paramType`	1125	4	−0.0000	0.0030
SPP	`nestingDepth`	1125	4	0.0071	0.0088

PCH clusters all executions globally; SPP uses its 1,125-execution view (Section 6.3). For PI, n counts balanced pairs (Section 6.2).

Table 12 consolidates this attack across all three objectives: K-Means recovers essentially nothing, with ARI within ±0.003 and NMI below 0.03 for every DUO, PI, and SPP target except one. NMI values are interpreted against the null simulation in Section 6.8: the slightly higher ICD-10 NMI (0.0281) lies within the range of a signal-free ten-class clustering with the same chapter sizes ($p = 0.15$), and permuting the chapter labels against the observed clusters gives the same reading (debiased NMI 0.0044, $p = 0.08$). The exception is the SPP `nestingDepth` axis, whose ARI of 0.007 is tiny in magnitude but permutation-significant ($p_{\text{ARI}} = 0.005$). It mirrors the `medium/deep` failure of Section 6.6: clustering the same executions on the residual gas (access gas minus the padding target) returns the axis to ARI −0.0004 ($p = 0.60$), and no other axis exceeds ARI 0.003 on the residual (`paramType` reaches 0.0026, $p = 0.04$). The grouping therefore follows the per-policy random draws of the padding. With that qualification, the unsupervised attacker agrees with the supervised verdict as a second, label-free attack family on the same dataset (Figure 2).

Removing sample-size confounds. The PI clustering inputs are rebalanced into 30-vs-30 contrasts and permutation-debiased as described in Section 6.2, so the near-zero ARI/NMI is not a degenerate-measurement artifact: over the 127 balanced pairs the mean ARI is



Figure 2. Unsupervised K-Means clustering across three objectives: ARI (top) and NMI (bottom) per target.

0.002, the mean debiased NMI 0.001, and 6 pairs (5%) reach nominal significance, the fraction expected under the null at $\alpha = 0.05$. The conclusion matches the supervised PI verdict (Section 6.5), and the ablation (Section 6.9) confirms the clustering device itself is live: on the unpadded transcript the same balanced pairs cluster with a mean debiased NMI 0.67 higher, and the signal disappears once padding is applied.

6.8. Sample-Size Sensitivity of NMI

The clustering cross-check (Section 6.7) treats ARI as the primary conservative indicator because NMI can take moderately high values under small samples. To justify that choice, we quantify how *finite-sample effects* alone can inflate NMI even when there is *no* discriminative signal, using an idealized null in which all classes are statistically indistinguishable.

Null hypothesis. We sample every class from the *same* distribution, $z_i \sim \mathcal{N}(0, I_d)$ for all classes, where I_d is the d-dimensional identity covariance and $d = 2$ matches the two gas features the clustering uses. Since no class carries class-specific information, a clustering algorithm should not recover class membership, and both NMI and ARI should concentrate around 0 as the sample size grows.

Simulation. For each configuration we run $T = 1000$ independent trials. In each trial we (i) generate G classes with n i.i.d. samples per class from $\mathcal{N}(0, I_d)$, for $N = Gn$ samples in total, assigning each sample a class label by construction order (these labels are independent of the sample values, since all classes share one distribution), (ii) standardize the pooled data and apply K-Means with $K = G$ and ten initializations, as in the evaluation, and (iii) score the predicted assignments against these assigned class labels with NMI and ARI. We vary n while holding G fixed, and report the trial means $\overline{\text{NMI}}$ and $\overline{\text{ARI}}$. We consider a **PI-like** binary structure ($G = 2$, $n \in \{3, 10, 50, 100\}$), a five-class structure ($G = 5$,

$n \in \{3, 10, 100\}$), and a ten-class structure ($G = 10$, $n \in \{3, 10, 77\}$) together with the observed ICD-10 chapter sizes (49–135 executions per chapter, $N = 770$).

Results. Table 13 shows that, although all classes are indistinguishable by construction, **NMI is noticeably positive at very small n but collapses toward zero as n grows**, whereas **ARI stays near zero throughout**. For the binary structure, $\overline{\text{NMI}}$ falls from 0.2074 at n=3 (N=6) to 0.0420 at n=10, and to 0.0072/0.0034 at n=50/100. The same trend holds with more classes, at a higher level for every n: with five classes $\overline{\text{NMI}}$ goes from 0.4152 (n=3) to 0.0102 (n=100), and with ten classes from 0.5346 (n=3) to 0.0239 (n=77). For the observed ICD-10 chapter sizes, a separate null run that averages each trial over 20 K-Means seeds, as the evaluation does, gives a null NMI with mean 0.0246 and 95th percentile 0.0302; the observed ICD-10 NMI of 0.0281 is reached in 145 of 1000 null trials ($p = 0.15$).

Table 13. Sample-size sensitivity of NMI and ARI under the null hypothesis where all classes are i.i.d. sampled from $\mathcal{N}(0, I_d)$.

Structure	n (per class)	G	N	$\overline{\text{NMI}}$	$\overline{\text{ARI}}$
PI-like	3	2	6	0.2074	0.0127
PI-like	10	2	20	0.0420	0.0008
PI-like	50	2	100	0.0072	−0.0003
PI-like	100	2	200	0.0034	−0.0004
5-class	3	5	15	0.4152	−0.0034
5-class	10	5	50	0.1226	0.0003
5-class	100	5	500	0.0102	−0.0000
10-class	3	10	30	0.5346	−0.0009
10-class	10	10	100	0.2087	0.0003
10-class	77	10	770	0.0239	−0.0001
ICD-10 sizes	49–135	10	770	0.0244	−0.0001

T=1000 trials per row, one K-Means fit with ten initializations per trial; G classes, $N = Gn$ samples.

6.9. Gas-Padding Ablation

Under the full defense configuration, the modeled attacks stay below the leakage criterion for all three privacy objectives, except for the two failures traced to the padding target. Because calldata length, proof size, and event logs are constant across the evaluated policies by design (Section 6.1), any policy-dependent signal can reach the observer only through gas usage. The ablation therefore measures how much of this signal randomized gas padding removes.

We compare two protection configurations on the same selected policy set, differing only in whether randomized gas padding is enabled. **B1** uses the fixed circuit, commitment-only interface, and TPS execution profile, with gas padding disabled for both policy commitment and access transactions. **B2** enables randomized gas padding for both transaction types, yielding the full defense configuration used in the evaluation. The policy subset is selected deterministically from the evaluation corpus. Each content label and structural bucket is represented by at least 70 executions from at least 8 distinct policies. All policies in the depth layer (Section 6.3) are retained.

The paired leakage metric is

$$\Delta_{\text{leak}} = |\text{AUC}_{\text{B1}} - 0.5| - |\text{AUC}_{\text{B2}} - 0.5|, \tag{19}$$

where positive values mean padding moved the attacker closer to chance. For each task we compute Δ_{leak} on each of 30 paired train/test splits, in which B1 and B2 share the same test policies, and report its mean with a 95% percentile-bootstrap interval over the splits

(labelled CI in Table 14 and Figure 3). Table 14 is separated from the privacy verdicts: the verdicts answer whether the full defense configuration is private; this ablation attributes that privacy to a specific defense layer and reports its gas cost.

Table 14. Gas-padding ablation on Polygon Amoy: B1 (no padding) versus B2 (deployed randomized gas padding) on the same execution subset; supervised rows are the largest-signal contrasts per experiment (per axis for SPP).

Attack	Target	B1 (unpad.)	B2 (pad.)
Supervised (RF AUC; Δ_{leak} with 95% interval)			
SPP	`structureType trivial/and_or_mixed`	0.902	0.490
	$\Delta_{\text{leak}} = 0.347$ CI$[0.334, 0.360]$		
SPP	`predicateCombo single/few`	0.844	0.494
	$\Delta_{\text{leak}} = 0.301$ CI$[0.287, 0.314]$		
PCH	DUO HMB	0.876	0.495
	$\Delta_{\text{leak}} = 0.337$ CI$[0.319, 0.352]$		
PI	`controlled_difference` †	0.676	0.446
	$\Delta_{\text{leak}} = 0.088$ CI$[0.062, 0.111]$		
Unsupervised (NMI, B1−B2 with bootstrap 95% interval)			
PCH	DUO terms	0.349 [0.316, 0.407]	
SPP	logic structure	0.102 [0.018, 0.138]	
PI	balanced pairs (debiased NMI)	0.670 [0.591, 0.743]	
Gas cost (mean over the subset, B1 → B2)			
Commit gas		219,737	304,098 (+38.4%)
Access gas		723,462	1,056,674 (+46.1%)

† Positive control: the least similar pairs by construction, excluded from the PI verdict.

Padding removes otherwise-recoverable signal. Figure 3 plots supervised tasks by their deviation from chance, $|\text{AUC} - 0.5|$, without padding (B1, horizontal axis) and with padding (B2, vertical axis); a point below the diagonal is a task whose attacker moved closer to chance once padding was applied. Without padding, 21 tasks deviate from chance by more than 0.15; with padding, all 21 fall below this level, and across all tasks the median deviation drops from 0.137 to 0.038. By family, the median Δ_{leak} is +0.173 for SPP and +0.058 for PCH. For PI it is −0.011, because three of its four tasks already deviate from chance by at most 0.08 without padding; the fourth, the positive control, drops from AUC 0.68 to 0.45.

Source of the unpadded signal. The B1 result deserves a precise reading, because the fixed circuit already makes the policy logic invisible to gas: the Groth16 verifier performs the same work for every proof, and the access-control path is the same for every policy. The unpadded access gas of the 1,119 executions spans only 723,288–723,564 gas (one cold-start first call at 741,276 aside), in steps of 12 gas, and eight of the ten ICD-10 chapter tasks stay near chance on it (random-forest AUC 0.455–0.536); the exception, ch6, has only 9 policies in the subset.

Twelve gas is the price difference between a zero and a non-zero calldata byte set by Ethereum Improvement Proposal 2028 (EIP-2028), and decoding the calldata of the transactions shows where the variation comes from: the requester pseudonym id_{req}. In the evaluation, a requester who presents a profile credential is identified by a Poseidon-derived pseudonym that fills the 256-bit word, whereas a requester of an ontology-only policy carries a 128-bit random pseudonym whose upper 16 bytes are zero; the 192-gas difference between the two encodings is exactly the gap between the two clusters of B1 gas values, and whether a policy activates the profile predicate is correlated with its DUO terms, its structure type, and its parameter bucket. The remaining spread of 84 gas within

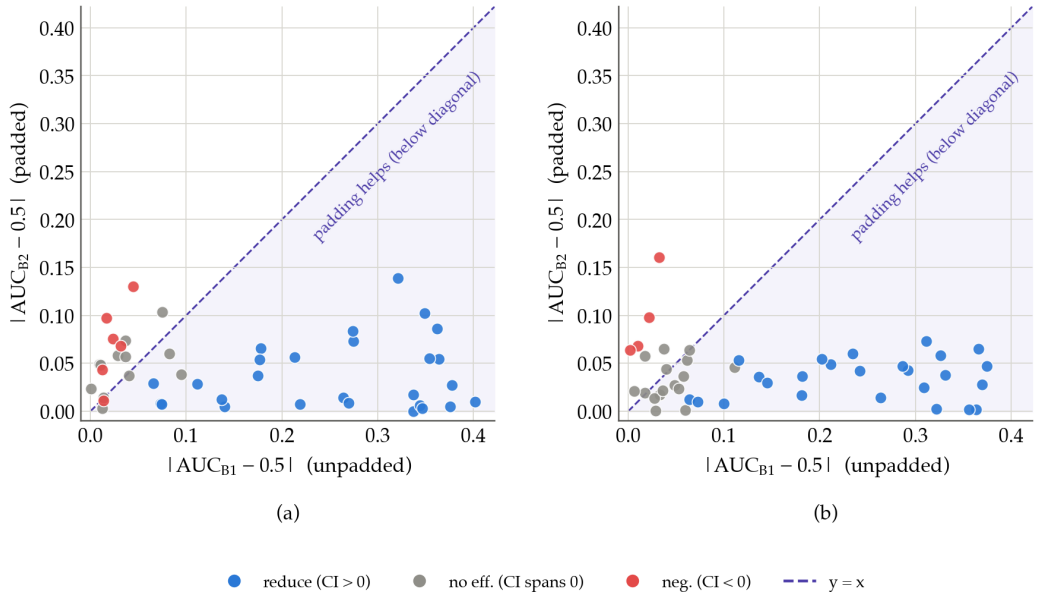


Figure 3. Gas-padding ablation over supervised tasks: (**a**) RF; (**b**) LR.

each cluster comes from the zero-byte counts of the proof elements, the policy and request identifiers, and the epoch, none of which depends on the policy.

The unpadded leak is therefore not the policy logic but a sub-kilogas, deterministic calldata-pricing artifact of how one public transaction argument is encoded; it is a real channel for the observer nonetheless, and it is the class of channel that padding is designed to absorb. The padding routine counts the zero and non-zero calldata bytes of the transaction and tops its gas up to the drawn target, so this 192-gas difference is compensated exactly; the overhead that remains after padding varies by less than 100 gas (Section 6.4), and randomizing the target by ±10% (±105,000 gas) masks it. Encoding every pseudonym at full width would close this particular channel without padding; randomized gas padding remains the more general defense because it also absorbs deterministic gas differences caused by other public encodings or execution effects.

As summarized in Table 14, the clustering attacker shows the same pattern on a second, label-free attack family: the debiased NMI of the balanced PI pairs is 0.67 higher under B1 than under B2, and the DUO-term and logic-structure clusterings lose 0.35 and 0.10 of NMI, each with a policy-group bootstrap interval excluding zero.

The effect is conditional, and we report it as such: the 12 tasks with no detectable effect sit in the lower-left of the figure, where B1 is already near chance, and the 6 tasks on which padding *raises* the deviation (negative Δ_{leak}, shown in red) are cases where B1 is at chance and the padded transcript's remaining deviation is the per-policy padding-target draw examined in Sections 6.4 and 6.6 (ICD ch5 and `nestingDepth` `medium/deep` among them). These per-task counts are descriptive, not multiplicity-corrected; the causal claim rests on the direction and magnitude of the shift where a signal is present, corroborated by the independent clustering attacker.

Cost. Padding is not free: over the subset it raises mean commit gas from 219,737 (B1) to 304,098 (B2), a 38.4% increase, and mean access gas from 723,462 to 1,056,674, a 46.1% increase. The padded access cost is set by the padding band: the TPS draws the target uniformly from 945,000–1,155,000 gas, a band placed above the bare access cost on the target network (about 723,000 gas on Amoy, as measured in B1) so that the whole band stays above real usage. At the 30 gwei tip used throughout, one padded access transaction costs about 0.032 POL (the native token of Polygon) and one commit about 0.009 POL; the

overhead is therefore a fixed per-transaction premium of roughly 330,000 gas on access and 85,000 on commit, independent of transaction volume, and it does not grow with policy complexity, because neither the bare cost nor the band depends on the policy.

Table 15 gives the single-point cost characterization of the evaluated prototype configuration at DEPTH $= 64$. The circuit compiles to a rank-1 constraint system (R1CS) of 81,740 constraints; across the 1,773 Polygon Amoy executions, mean witness and proving times on the two-core capture host are 763.6 ms and 2,884.2 ms, respectively. Gas, logs, calldata lengths, and transaction objects are taken from the same executed dataset, whose transactions use the context-bound five-signal statement. Within these costs, binding the program length (Section 5.3) adds 88 constraints and no public input, and binding the authorization context adds 605 constraints and one public input. At Amoy's gas schedule, the context binding adds 64,022 gas per access and recording the bound context on-chain adds 68,946 gas, together 18% of the mean unpadded access cost; because the unpadded cost stays below the padding band, the binding does not change the padded cost.

Table 16 reports two offline benchmarks of circuit capacity, each with 30 verified proofs per configuration on an Apple M1 using the rapidsnark Groth16 prover. With $N = 13$, $M = 11$, and eight Merkle levels fixed, increasing DEPTH from 32 to 128 raises the circuit from 64,945 to 134,009 constraints and the median proving time from 0.411 to 0.734 s. With DEPTH $= 64$ fixed, increasing the ontology-tree depth from 4 to 16 raises the circuit from 73,876 to 97,468 constraints and the median proving time from 0.512 to 0.571 s for a five-slot ontology-heavy policy. Every variant retains five public signals, so enlarging either capacity changes the prover artifacts but not the verifier interface.

Table 17 evaluates policy content within the evaluated circuit (DEPTH $= 64$, eight Merkle levels). Across simple, ontology-heavy, profile-heavy, and max-mixed policies, median witness time is 0.335–0.354 s and median proving time is 0.484–0.545 s, despite the encoded programs ranging from 3 to 61 tokens and activating from one to all 11 production predicate slots. The larger end-to-end time of the profile-backed policies (about 0.64 s) comes from the benchmark harness, which issues and signs a fresh profile credential for every request; in deployment the issuer signs a credential once, so this cost is not incurred per request. The fixed-circuit witness, prove, and verify work remains stable. Under load, throughput stays between 0.79 and 0.94 proofs/s as concurrency rises from one to four, while median latency grows from 1.2 to 4.1 s; Section 7 converts the measured capacity into requests per day.

Table 15. Single-point cost profile of the evaluated prototype configuration at DEPTH $= 64$. Gas, logs, calldata, and local prover timings are summarized over the 1,773 Polygon Amoy executions.

Metric	Value / source
Circuit size	81,740 R1CS constraints; 80,884 wires; DEPTH $= 64$
Public interface	4 public inputs plus 1 output
Padded on-chain gas	Commit 304,167; access 1,057,283 mean gas
Event logs	Commit: 1 log / 180 bytes; access: 2 logs / 296 bytes
Calldata length	Commit 196 bytes; access 612 bytes (constant across policies)
Local witness time	763.6 ms mean
Local prove time	2,884.2 ms mean

7. Discussion

Privacy findings. The evaluation supports a scoped claim: against the passive public-chain observer, TPS-ZKPI provides empirical resistance to policy leakage. Apart from the two failures discussed next, every supervised task stays below the leakage criterion, and unsupervised clustering shows no recoverable structure beyond one small, permutation-significant nesting-depth ARI.

Table 16. Offline benchmarks of circuit capacity (30 verified proofs per row; every variant keeps $N = 13$, $M = 11$, and five public signals).

Parameter	Capacity	Constraints	Wires	WASM (MB)	zkey (MB)	Witness	Prove	Verify
Program capacity: Merkle levels fixed at 8; mixed policy								
DEPTH	32	64,945	64,441	8.6	38.7	285 (343)	411 (562)	11 (16)
DEPTH	64	81,740	80,884	8.7	52.2	350 (414)	521 (761)	11 (22)
DEPTH	128	134,009	132,449	10.1	87.5	535 (682)	734 (984)	11 (15)
Ontology capacity: DEPTH $= 64$*; five-slot ontology-heavy policy*								
Merkle levels	4	73,876	73,052	8.6	48.3	352 (415)	512 (605)	11 (17)
Merkle levels	8	81,740	80,884	8.7	52.2	347 (354)	499 (539)	11 (16)
Merkle levels	12	89,604	88,716	8.7	56.0	358 (395)	552 (632)	10 (15)
Merkle levels	16	97,468	96,548	8.7	59.9	370 (456)	571 (713)	10 (13)

Timing cells report median (p95) in ms; WASM and zkey are the sizes of the witness generator and the proving key.

Table 17. Policy-content and workload scalability of the evaluated circuit (DEPTH $= 64$, eight Merkle levels) on one Apple M1 host using rapidsnark.

(a) Policy content

			Time (ms): median (p95)			
Policy	Tokens	Slots	Witness	Prove	Verify	End-to-end
Simple	3	1	354 (418)	545 (746)	12 (14)	925 (1170)
Ontology-heavy	19	5	349 (391)	506 (550)	10 (12)	880 (939)
Profile-heavy	31	1	337 (349)	492 (522)	10 (11)	1479 (1516)
Max-mixed	61	11	335 (346)	484 (537)	10 (10)	1468 (1538)

(b) Load

Concurrency c	Throughput (proofs/s)	Latency (ms): median (p95)
1	0.805	1230 (1659)
2	0.787	2492 (3805)
4	0.936	4113 (5854)

Tokens: program length ℓ (at most DEPTH $= 64$); slots: external-predicate slots the policy uses (of $M = 11$). End-to-end time also includes off-circuit evidence preparation, which for the profile-backed policies includes the harness's per-request issuance of a signed credential (about 0.64 s). Latency is each request's processing time while up to c requests run at once; queueing is excluded. All timings follow five warm-up requests.

The two tasks that meet the criterion (one ICD-10 chapter and the `nestingDepth medium/deep` contrast) are reported as failures, and both are carried by the randomly drawn padding target: subtracting the logged padding target from the access gas returns each to PASS, and replacing the gas features by policy-independent draws reproduces such failures in four of ten replicates (Sections 6.2–6.6). Such false failures are a property of the criterion on a corpus in which coverage-layer policies are executed once, so the thirty seeds resample the same random draws; we keep the criterion as stated and document the diagnostic, since raising the significant-seed or AUC threshold far enough to absorb these cases would also weaken it against genuine leaks.

The gas-padding ablation (Section 6.9) attributes the gas-channel protection to randomized gas padding: without it the same attacks recover policy content and structure with AUCs up to 0.90. The commitment-only interface and the fixed circuit keep the policy logic itself out of the transcript, but they do not remove every deterministic gas difference that correlates with the policy: the ablation exhibits one that comes not from the policy logic but from the byte encoding of a public transaction argument, and padding absorbs it along with any other such difference. Together with fixed-length policy encoding, the reusable interpreter, and commitment binding, these mechanisms limit the extent to which public-chain observers infer hidden policy content or structural properties from authorization transactions.

The cost of padding. Within the evaluated scope, the residual cost is the gas overhead of randomized gas padding (Section 6.9). The overhead is a fixed premium per transaction (roughly 330,000 gas on access and 85,000 on commit at the band used here), so it scales linearly with transaction volume and does not compound: at the 30 gwei tip of the evaluation, a padded access transaction costs about 0.032 POL against 0.022 POL unpadded. Both the bare cost and the band are policy-independent: the fixed circuit gives every proof the same verification work and the same five public signals, and the prover-side cost of larger capacities or different policy contents (Tables 16 and 17) is paid off-chain.

The offline load benchmark in Table 17 rotates the four policy classes through 100 verified requests after five warm-ups at each concurrency level. On the evaluated Apple M1, the best observed batch throughput is 0.936 proofs/s at concurrency four, with median and p95 request latencies of 4.113 and 5.854 s; CPU contention makes the concurrency-two result slightly lower. At this throughput, one prover on this host handles about 80,900 requests per day, or about 53,900 if a third of its capacity is held in reserve; the estimate is conservative, because half of the rotated requests include the harness's credential issuance (Section 6.9). On-chain, each padded access costs 1,057,283 gas on average (about 0.032 POL at 30 gwei; Table 15), so the access cost grows linearly with the number of requests, and each policy commit adds a one-time 304,167 gas (0.009 POL).

The band must be positioned above the bare cost on the target network, which differs between networks (the same verifier costs 243,000 gas on a local EVM and 372,000 on Amoy, whose BN254 precompiles are priced higher); a deployment therefore calibrates the base target once per network, and the width of the band, $\pm 10\%$ here, determines how large a residual difference the random target can mask.

TPS trust and credential provenance. The TPS trust assumption is the main architectural tradeoff. Moving confidentiality-sensitive proof generation to the TPS keeps policy-bound proving material away from requesters and public smart contracts while preserving on-chain verifiability, but it shifts part of the confidentiality burden to the service layer (Section 4). It also affects liveness and accountability: the TPS can refuse to produce a proof, and such a refusal may be indistinguishable from a legitimate denial at the on-chain interface.

Attribute origin is only partially delegated to the TPS. For the profile-backed DUO terms, the circuit verifies the EdDSA-Poseidon signature described in Section 5 and derives eight policy attributes from the seven signed claims. A compromised TPS cannot modify the signed claims, policy commitment, requester pseudonym, resource, validity interval, or credential identifier without invalidating the signature. The implemented relation rejects a non-committed issuer, an invalid signature under the committed key, an expired or premature credential, and substitutions of any signed field. However, because the TPS receives the holder secret and credential witness in the current workflow, holder binding does not protect against a malicious TPS reusing otherwise valid evidence within its signed scope.

Issuer governance and deployment path. The remaining trust anchor is therefore issuer governance. The prototype binds the proof to a policy-selected issuer-key commitment, but it has no independent registry establishing that this key belongs to an accredited healthcare authority, and it does not check credential revocation.

A concrete deployment can add issuer governance without changing the Boolean interpreter: a governed registry would publish schema-qualified issuer keys and their validity epochs, while an issuer would sign a domain-separated credential containing a holder-key commitment, claim vector, authorization scope, validity interval, and unique credential identifier. The circuit would verify the credential signature and a Merkle proof that the issuer key is active in the registry, while continuing to derive policy attributes

from the signed claims. The holder would authorize each request by signing the context digest and request epoch, preventing a TPS that observes one credential presentation from generating proofs for fresh sessions; an epoch-specific sparse-Merkle or accumulator root would provide a private non-revocation witness for the credential identifier.

The registry and revocation roots can occupy fixed-format public or committed parameter slots, preserving the Boolean interpreter and the shape of the policy-independent public interface, although these checks require a revised circuit and new proving and verification keys. Existing healthcare credentials using Ed25519 or the Elliptic Curve Digital Signature Algorithm (ECDSA) require either higher-cost in-circuit verification or an additional SNARK-friendly credential; TPS re-signing cannot establish issuer-origin assurance.

Policy and context binding. As described in Section 5, the policy commitment covers both the padded encoding P_{enc} and its active length ℓ. Because ℓ also determines how many tokens the interpreter evaluates, an authorizing proof for h cannot be based on a shorter valid prefix. This integrity guarantee is separate from our privacy evaluation, which measures whether public-chain artifacts reveal information about the hidden policy.

The authorization context is likewise part of the proved relation. The circuit constrains the public digest d_χ to the requester pseudonym, resource, and nonce used in the witness, and the ACM recomputes the same digest from the transaction parameters. Consequently, changing any of these values invalidates the request.

The guarantee is scoped to the bound tuple within one ACM deployment. The digest does not include the transaction submitter or a chain/contract domain, so exact-context front-running and cross-deployment reuse require separate deployment-level controls.

Future directions. Three directions follow. First, adding an explicit chain/contract domain and an authenticated submission channel would extend context binding beyond a single deployment and reduce front-running risk. Second, accountable proof obligations (for example, on-chain commitments to serve requests together with timeout-with-evidence mechanisms) would make proof refusals attributable. Third, the reduced-trust TPS instantiations of Section 4 (collaborative-SNARK or TEE-backed proving) would relax the trust concentration itself, including the attribute-origin limitation; realizing and evaluating them is left to future work.

8. Conclusion

This paper presented TPS-ZKPI, a Trusted Proof Service-assisted zero-knowledge approach to policy-private consent enforcement on public blockchains. In TPS-ZKPI, the data provider commits to a hidden consent policy on-chain, the TPS proves in zero knowledge that a requester satisfies this policy, and a smart contract verifies the proof. Each policy is compiled into a fixed-length program for a reusable zero-knowledge policy interpreter, so all policies share one verification interface, and randomized gas padding masks the remaining gas differences. We evaluated TPS-ZKPI on the Polygon Amoy testnet with supervised and unsupervised attacks in which a passive public-chain observer tried to infer hidden policy properties from public-chain metadata. The results showed that nearly all supervised tasks stayed below our leakage criterion, that both failures traced to chance correlations with the padding, and that unsupervised clustering found no policy structure beyond the same padding effect. Furthermore, the gas-padding ablation showed that, without padding, the same attacks often recovered policy content and structure, which confirmed that padding protects the gas channel at a fixed gas cost per transaction. Overall, this paper showed empirically that consent enforcement on a public blockchain can resist the modeled metadata attacks, provided that the TPS follows its workflow.

Author Contributions: Withheld for review.

Funding: Withheld for review.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: TODO.

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

TPS-ZKPI	Trusted Proof Service-assisted Zero-Knowledge Policy Interpreter
TPS	Trusted Proof Service
ACM	Access-Control Manager
ZKP	Zero-Knowledge Proof
SNARK	Succinct Non-interactive Argument of Knowledge
TEE	Trusted Execution Environment
RPN	Reverse Polish Notation
PCH	Policy Content Hiding
PI	Policy Indistinguishability
SPP	Structure and Parameter Privacy
DUO	Data Use Ontology
ICD-10	International Classification of Diseases, 10th Revision
OBI	Ontology for Biomedical Investigations
GA4GH	Global Alliance for Genomics and Health
CP-ABE	Ciphertext-Policy Attribute-Based Encryption
PRE	Proxy Re-Encryption
AUC	Area Under the (ROC) Curve
ARI	Adjusted Rand Index
NMI	Normalized Mutual Information
LR	Logistic Regression
RF	Random Forest
ABE	Attribute-Based Encryption
ABAC	Attribute-Based Access Control
MPC	Multi-Party Computation
HE	Homomorphic Encryption
SSI	Self-Sovereign Identity
IoMT	Internet of Medical Things
XACML	eXtensible Access Control Markup Language
SAML	Security Assertion Markup Language
EdDSA	Edwards-curve Digital Signature Algorithm
ECDSA	Elliptic Curve Digital Signature Algorithm
EVM	Ethereum Virtual Machine
EIP	Ethereum Improvement Proposal
R1CS	Rank-1 Constraint System
MONDO	Mondo Disease Ontology
HANCESTRO	Human Ancestry Ontology
SNOMED CT	SNOMED Clinical Terms

References

1. Kakarlapudi, P.V.; Mahmoud, Q.H. A systematic review of blockchain for consent management. *Healthcare* **2021**, *9*, 137. https://doi.org/10.3390/healthcare9020137.

2. Welzel, C.; Ostermann, M.; Smith, H.L.; Minssen, T.; Kirsten, T.; Gilbert, S. Enabling secure and self determined health data sharing and consent management. *npj Digital Medicine* **2025**, *8*, 560. https://doi.org/10.1038/s41746-025-01945-z.
3. Li, K.; Sai, A.R.; Urovi, V. Do you need a blockchain in healthcare data sharing? A tertiary review. *Exploration of Digital Health Technologies* **2024**, *2*, 101–123. https://doi.org/10.37349/edht.2024.00014.
4. Béres, F.; Seres, I.A.; Benczúr, A.A.; Quintyne-Collins, M. Blockchain is watching you: Profiling and deanonymizing Ethereum users. In Proceedings of the 2021 IEEE International Conference on Decentralized Applications and Infrastructures (DAPPS). IEEE, 2021, pp. 69–78. https://doi.org/10.1109/DAPPS52256.2021.00013.
5. Li, K.; Lohachab, A.; Dumontier, M.; Urovi, V. Privacy preservation in blockchain-based healthcare data sharing: A systematic review. *Peer-to-Peer Networking and Applications* **2025**, *18*, 1–53. https://doi.org/10.1007/s12083-025-02148-9.
6. Lawson, J.; Cabili, M.N.; Kerry, G.; Boughtwood, T.; Thorogood, A.; Alper, P.; Bowers, S.R.; Boyles, R.R.; Brookes, A.J.; Brush, M.; et al. The Data Use Ontology to streamline responsible access to human biomedical datasets. *Cell Genomics* **2021**, *1*, 100028. https://doi.org/10.1016/j.xgen.2021.100028.
7. Rondelet, A.; Zajac, M. Zeth: On Integrating Zerocash on Ethereum. arXiv:1904.00905, 2019, [arXiv:cs.CR/1904.00905]. https://doi.org/10.48550/arXiv.1904.00905.
8. Williamson, Z.J. The AZTEC Protocol. https://github.com/AztecProtocol/AZTEC, 2018. Technical whitepaper. Accessed: 2026-09-20.
9. Nadler, M.; Schär, F. Tornado Cash and blockchain privacy: a primer for economists and policymakers. *Federal Reserve Bank of St. Louis Review* **2023**, *105*, 122–136. https://doi.org/10.20955/r.105.122-136.
10. Ibraimi, L.; Petkovic, M.; Nikova, S.; Hartel, P.; Jonker, W. Ciphertext-policy attribute-based threshold decryption with flexible delegation and revocation of user attributes. Technical report, Centre for Telematics and Information Technology, University of Twente, 2009. https://research.utwente.nl/files/5098396/Technical_Report.pdf.
11. Ullah, S.S.; Oleshchuk, V.; Pussewalage, H.S.G. A survey on blockchain envisioned attribute based access control for internet of things: Overview, comparative analysis, and open research challenges. *Computer Networks* **2023**, *235*, 109994. https://doi.org/10.1016/j.comnet.2023.109994.
12. Zhou, L.; Diro, A.; Saini, A.; Kaisar, S.; Hiep, P.C. Leveraging zero knowledge proofs for blockchain-based identity sharing: A survey of advancements, challenges and opportunities. *Journal of Information Security and Applications* **2024**, *80*, 103678. https://doi.org/10.1016/j.jisa.2023.103678.
13. Yang, X.; Liu, J. HA-Med: A Blockchain-Based Solution for Sharing Medical Data with Hidden Policies and Attributes. *IET Information Security* **2024**, *2024*, 2498245. https://doi.org/10.1049/2024/2498245.
14. Ding, X.; Liu, Y.; Ning, J.; Chen, D. Blockchain-Enhanced Anonymous Data Sharing Scheme for 6G-Enabled Smart Healthcare With Distributed Key Generation and Policy Hiding. *IEEE Journal of Biomedical and Health Informatics* **2026**, *30*, 666–675. https://doi.org/10.1109/JBHI.2025.3550261.
15. World Health Organization. International Statistical Classification of Diseases and Related Health Problems (10th Revision). https://icd.who.int/browse10/2019/en, 2019. Accessed: 2026-07-08.
16. Dyke, S.O.; Philippakis, A.A.; Rambla De Argila, J.; Paltoo, D.N.; Luetkemeier, E.S.; Knoppers, B.M.; Brookes, A.J.; Spalding, J.D.; Thompson, M.; Roos, M.; et al. Consent codes: upholding standard data use conditions. *PLoS Genetics* **2016**, *12*, e1005772. https://doi.org/10.1371/journal.pgen.1005772.
17. Bethencourt, J.; Sahai, A.; Waters, B. Ciphertext-policy attribute-based encryption. In Proceedings of the 2007 IEEE Symposium on Security and Privacy (SP'07). IEEE, 2007, pp. 321–334. https://doi.org/10.1109/SP.2007.11.
18. Ateniese, G.; Fu, K.; Green, M.; Hohenberger, S. Improved proxy re-encryption schemes with applications to secure distributed storage. *ACM Transactions on Information and System Security (TISSEC)* **2006**, *9*, 1–30. https://doi.org/10.1145/1127345.1127346.
19. Pei, H.; Yang, P.; Li, W.; Du, M.; Hu, Z. Proxy Re-Encryption for Secure Data Sharing with Blockchain in Internet of Medical Things. *Computer Networks* **2024**, *245*, 110373. https://doi.org/10.1016/j.comnet.2024.110373.
20. Zhou, H.; Si, X.M.; Zhao, B.; Zhang, Y.; Qu, Y. Secure electronic medical records sharing scheme based on blockchain by using proxy re-encryption. *Peer-to-Peer Networking and Applications* **2024**, *17*, 4197–4209. https://doi.org/10.1007/s12083-024-01775-y.
21. Groth, J. On the Size of Pairing-Based Non-interactive Arguments. In Proceedings of the Advances in Cryptology – EUROCRYPT 2016. Springer, 2016, pp. 305–326. https://doi.org/10.1007/978-3-662-49896-5_11.
22. Sasson, E.B.; Chiesa, A.; Garman, C.; Green, M.; Miers, I.; Tromer, E.; Virza, M. Zerocash: Decentralized anonymous payments from Bitcoin. In Proceedings of the 2014 IEEE Symposium on Security and Privacy. IEEE, 2014, pp. 459–474. https://doi.org/10.1109/SP.2014.36.
23. Wu, N.; Xu, L.; Zhu, L. A blockchain based access control scheme with hidden policy and attribute. *Future Generation Computer Systems* **2023**, *141*, 186–196. https://doi.org/10.1016/j.future.2022.11.006.
24. Boneh, D.; Sahai, A.; Waters, B. Functional encryption: Definitions and challenges. In Proceedings of the Theory of Cryptography Conference. Springer, 2011, pp. 253–273. https://doi.org/10.1007/978-3-642-19571-6_16.

25. Secret Network. Secret Network Overview. https://docs.scrt.network/secret-network-documentation/overview-ecosystem-and-technology/secret-network-overview. Accessed: 2026-06-29.
26. Cheng, R.; Zhang, F.; Kos, J.; He, W.; Hynes, N.; Johnson, N.; Juels, A.; Miller, A.; Song, D. Ekiden: A platform for confidentiality-preserving, trustworthy, and performant smart contracts. In Proceedings of the 2019 IEEE European Symposium on Security and Privacy (EuroS&P). IEEE, 2019, pp. 185–200. https://doi.org/10.1109/EuroSP.2019.00023.
27. Buccafurri, F.; De Angelis, V.; Nardone, R. How can the holder trust the verifier? A CP-ABPRE-based solution to control the access to claims in a Self-Sovereign-Identity scenario. *Blockchain: Research and Applications* **2024**, *5*, 100196. https://doi.org/10.1016/j.bcra.2024.100196.
28. Madine, M.; Salah, K.; Jayaraman, R.; Yaqoob, I. Zero-knowledge proofs for anonymous authentication of patients on public and private blockchains. *Array* **2025**, *28*, 100590. https://doi.org/10.1016/j.array.2025.100590.
29. Shashidhara, R.; Chirakarotu Nair, R.; Kumar Panakalapati, P. Promise of Zero-Knowledge Proofs (ZKPs) for Blockchain Privacy and Security: Opportunities, Challenges, and Future Directions. *Security and Privacy* **2024**, *8*, e461. https://doi.org/10.1002/spy2.461.
30. Ardagna, C.A.; De Capitani di Vimercati, S.; Paraboschi, S.; Pedrini, E.; Samarati, P. An XACML-based privacy-centered access control system. In Proceedings of the First ACM Workshop on Information Security Governance, 2009, pp. 49–58. https://doi.org/10.1145/1655168.1655178.
31. Sheikhalishahi, M.; Tillem, G.; Erkin, Z.; Zannone, N. Privacy-preserving multi-party access control. In Proceedings of the 18th ACM Workshop on Privacy in the Electronic Society, 2019, pp. 1–13. https://doi.org/10.1145/3338498.3358643.
32. Gabizon, A.; Williamson, Z.J.; Ciobotaru, O. PLONK: Permutations over Lagrange-bases for Oecumenical Noninteractive arguments of Knowledge. Cryptology ePrint Archive, Paper 2019/953, 2019.
33. Campanelli, M.; Fiore, D.; Querol, A. LegoSNARK: Modular design and composition of succinct zero-knowledge proofs. In Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security, 2019, pp. 2075–2092. https://doi.org/10.1145/3319535.3339820.
34. Liu, T.; Zhang, Z.; Zhang, Y.; Hu, W.; Zhang, Y. Ceno: Non-uniform, segment and parallel zero-knowledge virtual machine. *Journal of Cryptology* **2025**, *38*, 17. https://doi.org/10.1007/s00145-024-09533-2.
35. Miller, T.; Alvarez, B.; Hoang, T. VTBC: Privatizing the Volume and Timing of Transactions for Blockchain Applications. In Proceedings of the 2023 32nd International Conference on Computer Communications and Networks (ICCCN). IEEE, 2023, pp. 1–10. https://doi.org/10.1109/ICCCN58024.2023.10230098.
36. Hu, R.; Ma, Z.; Li, L.; Zuo, P.; Li, X.; Wei, J.; Liu, S. An access control scheme based on blockchain and ciphertext policy-attribute based encryption. *Sensors* **2023**, *23*, 8038. https://doi.org/10.3390/s23198038.
37. Bernstein, D.J.; Duif, N.; Lange, T.; Schwabe, P.; Yang, B.Y. High-speed high-security signatures. *Journal of Cryptographic Engineering* **2012**, *2*, 77–89. https://doi.org/10.1007/s13389-012-0027-1.
38. WhiteHat, B.; Bellés, M.; Baylina, J. EIP-2494: Baby Jubjub Elliptic Curve. https://eips.ethereum.org/EIPS/eip-2494, 2020. Accessed: 2026-09-24.
39. Grassi, L.; Khovratovich, D.; Rechberger, C.; Roy, A.; Schofnegger, M. Poseidon: A New Hash Function for Zero-Knowledge Proof Systems. In Proceedings of the 30th USENIX Security Symposium, 2021, pp. 519–535.
40. Bandrowski, A.; Brinkman, R.; Brochhausen, M.; Brush, M.H.; Bug, B.; Chibucos, M.C.; Clancy, K.; Courtot, M.; Derom, D.; Dumontier, M.; et al. The Ontology for Biomedical Investigations. *PLoS ONE* **2016**, *11*, e0154556. https://doi.org/10.1371/journal.pone.0154556.
41. Jaccard, P. The Distribution of the Flora in the Alpine Zone. *New Phytologist* **1912**, *11*, 37–50. https://doi.org/10.1111/j.1469-8137.1912.tb05611.x.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.